

Hi,

I'm currently seeking new opportunities as a Java Developer. I have over 4.1 years of experience in Java, Spring Boot, and microservices, working on scalable backend solutions and API integrations. I'm passionate about clean code, problem-solving, and learning new technologies. Looking forward to contributing my skills in a dynamic team.

---

## Basic AWS Interview Questions

### 1. What is AWS?

Answer:

AWS (Amazon Web Services) is a cloud platform offering on-demand services like computing power, storage, databases, machine learning, and networking. It helps companies build and scale applications without maintaining physical servers.

---

### 2. What are the main services offered by AWS?

Answer:

Compute: EC2, Lambda, Elastic Beanstalk

Storage: S3, EBS, Glacier

Database: RDS, DynamoDB, Aurora

Networking: VPC, CloudFront, Route 53, API Gateway

Security: IAM, KMS, WAF, Shield

---

### 3. What is EC2?

Answer:

Amazon EC2 (Elastic Compute Cloud) provides virtual servers (instances) to run applications. You can scale instances up or down as needed.

Real-time use: Hosting microservices, APIs, and web apps.

---

### 4. What is S3 in AWS?

Answer:

Amazon S3 (Simple Storage Service) is an object storage service used to store files, backups, images, and logs.

Features:

High durability (99.999999999%)

Versioning

Lifecycle management

Static website hosting

---

5. What is IAM?

Answer:

IAM (Identity and Access Management) controls access to AWS resources.

Real-time use: Create users, roles, and policies to define who can access what.

---

6. What is an Availability Zone (AZ)?

Answer:

An AZ is a data center inside a region. AWS regions have multiple AZs for fault tolerance.

---

7. What is the difference between Region and AZ?

Region Availability Zone

Geographical area (e.g., ap-south-1) Specific data center inside a region

Has multiple AZs Belongs to one region only

---

8. What is Elastic Load Balancer (ELB)?

Answer:

ELB automatically distributes incoming traffic across multiple EC2 instances.

Types:

Application Load Balancer (HTTP/HTTPS)

Network Load Balancer (TCP/UDP)

Gateway Load Balancer (3rd party appliances)

---

9. What is Auto Scaling?

Answer:

Auto Scaling automatically increases or decreases the number of EC2 instances based on demand.

Real-time use: Maintain performance during high traffic.

---

10. What is CloudFormation?

Answer:

Infrastructure as Code (IaC) tool used to provision AWS resources automatically via YAML/JSON templates.

---

## Intermediate-Level AWS Interview Questions

11. What is the difference between EC2 and Lambda?

Feature EC2 Lambda

Type Virtual server Serverless compute

Pricing Pay per hour Pay per request

Scaling Manual/Auto Automatic

Use case Long-running apps Event-driven functions

---

12. What is VPC?

Answer:

A Virtual Private Cloud (VPC) allows you to create your own isolated network within AWS.

Includes:

Subnets, Route tables, Internet Gateway, NAT Gateway, Security groups.

---

13. What are Security Groups and NACLs?

Security Group Network ACL

Instance-level firewall Subnet-level firewall

Stateful Stateless

Allows inbound and outbound Controls inbound/outbound separately

---

14. What is Route 53?

Answer:

AWS's DNS (Domain Name System) service used for domain registration, routing, and health checking.

Example: Map domain → EC2 instance or S3 website.

---

15. What is CloudWatch?

Answer:

AWS monitoring and logging service.

Use: Track metrics, set alarms, and view application logs.

---

16. What is Elastic Beanstalk?

Answer:

Platform-as-a-Service (PaaS) to deploy applications quickly without managing servers manually.

---

17. What is AWS RDS?

Answer:

Managed relational database service supporting MySQL, PostgreSQL, Oracle, SQL Server, and Aurora.

Real-time use: For transactional applications (ERP, e-commerce, etc.)

---

18. What is DynamoDB?

Answer:

Fully managed NoSQL database that supports key-value and document data structures.

Use: Real-time apps like gaming leaderboards or IoT data.

---

19. What is AWS ECR and ECS?

ECR (Elastic Container Registry): Stores Docker images.

ECS (Elastic Container Service): Runs containers using EC2 or Fargate.

---

20. What is AWS Lambda used for in microservices?

Answer:

Lambda is used for running event-driven backend logic — e.g., user uploads image to S3 → Lambda resizes it.

---

 Advanced / Real-Time AWS Interview Questions

21. How does API Gateway work with Lambda?

Answer:

API Gateway acts as an entry point for API requests, invoking Lambda functions based on routes.

Flow:

Client → API Gateway → Lambda → Return Response

---

22. What is the difference between S3 and EBS?

Feature S3 EBS

Type Object storage Block storage

Attach Global access EC2-only

Use case Static files, backups Boot volumes, databases

---

23. What is AWS CloudTrail?

Answer:

CloudTrail records every API call or action made in your AWS account for auditing.

---

24. What is AWS KMS?

Answer:

Key Management Service for encrypting data across services.

---

25. What is the difference between Elastic Beanstalk and CloudFormation?

Beanstalk CloudFormation

Manages app deployment Manages full infrastructure

Simpler, auto-managed More control, custom templates

---

26. What is the use of SQS and SNS?

SQS (Simple Queue Service): Message queue for decoupled systems.

SNS (Simple Notification Service): Publishes messages to multiple subscribers (email, Lambda, etc).

Real-time use: Order service → sends message to SQS → inventory service processes it.

---

27. What is AWS Step Functions?

Answer:

Service to orchestrate multiple AWS services (Lambda, ECS, etc.) into a workflow.

---

28. What is the difference between public and private subnet?

Public Subnet Private Subnet

Has Internet Gateway No direct internet access

Used for web servers Used for DB servers

---

29. What is AWS EKS?

Answer:

Elastic Kubernetes Service — a managed Kubernetes service to deploy, manage, and scale containerized applications.

---

30. How do you secure AWS applications?

Answer:

Use IAM roles and least privilege.

Enable multi-factor authentication (MFA).

Encrypt data with KMS.

Monitor using CloudWatch and CloudTrail.

Enable VPC Security Groups and NACLs.

---

### Scenario-Based Questions

31. How do you handle high traffic in AWS?

Use Auto Scaling + ELB

Use CloudFront CDN

Use RDS read replicas

Optimize caching (ElasticCache / Redis)

---

32. Your EC2 instance is not reachable — how will you troubleshoot?

1. Check Security Group inbound rules.

2. Check NACL and Route Table.

3. Verify instance is running and has public IP.

4. Use Session Manager or EC2 serial console if SSH fails.

---

33. How do you ensure zero downtime deployment?

Use Blue/Green deployment in Elastic Beanstalk or ECS.

Use Load Balancer to route traffic between environments.

---

34. How to optimize AWS cost?

Use Reserved Instances / Spot Instances

Enable Auto Scaling

Delete unused EBS volumes

Use S3 lifecycle policies

---

35. How is data secured in S3?

Enable Server-Side Encryption

Configure Bucket Policies

Enable MFA Delete

Use Access Logs

---

Would you like me to prepare a focused set of 20 AWS interview questions specifically for your Java + Spring Boot microservices developer role (with real-time examples like S3 upload, Lambda trigger, etc.)?

That would make your preparation highly relevant.

In real time, you configure the Saga Pattern in microservices using Kafka (or another message broker) to coordinate steps through events instead of direct API calls.

Here's how it works in real implementation 📌

🔧 1. Setup Kafka Topics



Create topics for communication:

order-created

payment-success

payment-failed

inventory-success

inventory-failed

order-cancelled

Each microservice will publish and consume messages.

---

## 2. Order Service (Start Saga)

Creates order → saves status as PENDING.

Publishes event → order-created to Kafka.

```
kafkaTemplate.send("order-created", orderEvent);
```

---

## 3. Payment Service (Next Step)

Listens to order-created.

Deducts payment (local DB transaction).

Publishes event →

payment-success (if OK)

or payment-failed (if issue).

```
@KafkaListener(topics = "order-created")
public void processOrder(OrderEvent event) {
    try {
        paymentRepository.save(...);
        kafkaTemplate.send("payment-success", event);
    } catch (Exception e) {
        kafkaTemplate.send("payment-failed", event);
    }
}
```

}

---

#### 4. Inventory Service

Listens to payment-success.

Deducts stock → publishes inventory-success or inventory-failed.

---

#### 5. Compensation (Rollback Handling)

If inventory-failed happens → Saga triggers compensating actions:

Publish refund-payment → Payment Service refunds.

Publish cancel-order → Order Service updates order status to CANCELLED.

---

#### 6. Coordinator Options

You can manage Saga flow in two ways:

Choreography: Each service listens and reacts to events (via Kafka).  Common and simple.

Orchestration: A central Saga Orchestrator Service controls the flow (decides next steps).

---

#### Two Possible Communication Flows

When your Order Service wants to call the Payment Service, it depends on your architecture. There are two common patterns:

---

#### 1. Direct Service-to-Service Call (No API Gateway in Between)

Flow:

Order Service → (Client-side Load Balancer) → Payment Service Instance

How it works:

Order Service calls <http://payment-service/pay>.

Using Eureka + Spring Cloud LoadBalancer, the client (Order Service) finds all payment-service instances.

It chooses one instance (round-robin/random/custom) and calls it directly.

Pros:  Faster (no gateway hop)  
 Less overhead

Cons:  Each service must have load balancer + discovery client configured  
 Harder to secure (each service exposes its API)  
 Hard to monitor and manage cross-cutting concerns (logging, rate limit, auth)

Used when:  Internal microservice-to-microservice communication  
 Low-latency internal network

---

## 2. Via API Gateway

Flow:

Order Service → API Gateway → Payment Service Instance

How it works:

Order Service calls Gateway: <http://api-gateway/payment/pay>

Gateway uses service discovery to find Payment Service instances and load balance.

Gateway applies filters like:

Authentication

Logging

Rate Limiting

Retry, Circuit Breaker

Pros:  Centralized control (security, logging, auth)  
 No direct exposure of internal services  
 Easy to manage cross-cutting concerns

Cons:  Slightly more latency  
 Gateway is a single point of failure (unless scaled)

Used when: → All communication goes through a single entry point  
→ Services need consistent policies (auth/logging)

---

⚙ In Real-Time Microservices Architecture

In practice, both patterns are used together:

Communication Type Example Preferred Approach

External Client → Services Web app / Mobile app → Backend ✓ Go through API Gateway  
Internal Service → Service Order → Payment, Order → Inventory ✓ Direct service-to-service  
via client-side load balancing

---

When Order Service calls Payment Service directly:

#### 1 JWT Token Propagation

User logs in → gets JWT token.

Order Service receives token → passes it to Payment Service in request header.

Payment Service validates token.

#### 2 Service-to-Service Token (Machine-to-Machine)

Each service has a client ID + secret.

Order Service requests a token from Auth Server → sends it to Payment Service.

Payment Service validates token.

---

### ◆ What Are Memory Issues in Spring Boot Microservices?

Memory issues occur when your microservice consumes more heap or off-heap memory than available, leading to:

Slow response times

OutOfMemoryError

JVM GC (Garbage Collection) overhead

Service restarts or crashes under load

---

#### ♦ 1. Common Memory Issues (Real-Time)

Issue Description Example

🧠 Memory Leak Objects not released even after use Unclosed DB connections, thread pools, static references

♻️ High Garbage Collection (GC) Frequent GC cycles slow down app Large objects or many short-lived objects

🏗️ Improper Cache Usage Too much data stored in cache Using @Cacheable without eviction policy

📦 Large Object Retention Keeping large responses or lists in memory Large JSONs or Excel files

🔄 Thread Leaks Threads created but never destroyed Executor services without shutdown

📁 Improper Stream Handling InputStream not closed File or HTTP connection leaks

🔗 Too Many Microservice Instances Each service keeps its own memory footprint

Misconfigured container limits

---

#### ♦ 2. Real-Time Example

❌ Problem:

A Spring Boot microservice suddenly slows down and restarts automatically under high load. Logs show:

java.lang.OutOfMemoryError: Java heap space

✅ Root Cause:

The service cached large JSON payloads in a ConcurrentHashMap without eviction.

Memory kept increasing over time (memory leak).

✅ Fix:

Use Caffeine or Redis with proper TTL.

Add heap dump analysis using VisualVM or Eclipse MAT.

Configure JVM heap limits:

-Xms512m -Xmx1024m

---

### ♦ 3. Memory Optimization Techniques (Real Time)

#### Area Best Practice

- ♦ JVM Tuning Set proper heap size using -Xms and -Xmx.
- ♦ Monitoring Use Prometheus + Grafana or ELK to monitor memory usage.
- ♦ GC Logs Enable GC logs for analysis (-Xlog:gc\*).
- ♦ Close Resources Always close DB connections, streams, threads.
- ♦ Limit Cache Size Use cache TTL, max size (e.g. Caffeine: maxSize(1000)).
- ♦ Lazy Loading Fetch only required data from DB using pagination.
- ♦ Avoid Large Collections Don't store big lists in memory unnecessarily.
- ♦ Use Object Pools Reuse expensive objects (e.g., database connections).
- ♦ Memory Profiler Tools VisualVM, JProfiler, YourKit, Eclipse MAT.

---

### ♦ 4. Real-Time Tools for Memory Issue Analysis

#### Tool Purpose

VisualVM / JConsole Monitor heap usage, thread count, GC activity  
Eclipse Memory Analyzer (MAT) Analyze heap dumps for memory leaks  
Prometheus + Grafana Real-time JVM memory visualization  
Micrometer Collect JVM metrics in Spring Boot  
JProfiler / YourKit Advanced memory and CPU profiling

---

### ♦ 5. Real-Time Example Monitoring in Spring Boot

Expose JVM metrics using Spring Actuator:

management:

endpoints:

web:

exposure:

include: ["metrics", "prometheus"]

You can monitor:

jvm.memory.used

jvm.memory.max

system.cpu.usage

[tomcat.threads.active](#)

Prometheus or Grafana can alert if:

jvm\_memory\_used\_bytes > 0.9 \* jvm\_memory\_max\_bytes

---

💬 Interview Q&A (Real-Time Focus)

---

❶ What causes memory leaks in Spring Boot microservices?

Answer:

Memory leaks occur when objects are unintentionally kept in memory — e.g.:

Static variables holding large lists

Caches without eviction

Threads not terminated

Open InputStreams

Large HTTP responses not released

---

❷ How do you detect memory leaks?

Answer:

Use tools like VisualVM, JProfiler, or Eclipse MAT to:

Take a heap dump (jmap -dump:live,format=b,file=heap.bin <pid>)

Analyze retained objects and references

Identify objects preventing GC

---

❸ How to fix OutOfMemoryError in production?

Answer:

1. Increase heap size using JVM flags (-Xmx1024m)
2. Analyze heap dump to find the leak
3. Fix code — close resources, limit cache
4. Use monitoring to detect early
5. Implement load balancing or scaling

---

#### ④ How can you prevent memory leaks in Spring Boot?

Answer:

Use try-with-resources to close streams

Avoid static maps or singletons for large data

Add @CacheEvict or TTL in caching

Use object pooling for heavy resources

Set thread pools with proper limits

---

#### ⑤ How to monitor memory in microservices architecture?

Answer:

Use Spring Boot Actuator + Micrometer + Prometheus + Grafana:

Each microservice exposes memory metrics

Prometheus scrapes metrics

Grafana visualizes memory usage

Alerts trigger when usage exceeds threshold

---

#### ⑥ What is JVM Garbage Collection and how it affects performance?

Answer:

GC automatically reclaims memory from unused objects.



If the heap is too small or objects are too large, frequent GC pauses occur → causing slow response times.

---

7 How to optimize memory in microservices running on Docker or Kubernetes?

Answer:

Set container limits:

resources:

limits:

memory: "512Mi"

cpu: "500m"

Configure JVM memory inside container:

-XX:+UseContainerSupport

-Xmx400m

Use autoscaling based on memory usage

---

8 Difference between heap and non-heap memory?

Memory Type Purpose Example

Heap Memory Stores objects and data (managed by GC) Entities, Lists

Non-Heap Memory For JVM internals Class metadata, thread stacks, JIT cache

---

9 How can caching cause memory leaks?

Answer:

Improper caching (e.g. @Cacheable without TTL or max size) causes unlimited object growth in memory — leading to OutOfMemoryError.

---

10 What are GC tuning parameters you've used?

Answer:

Example JVM options:

-XX:+UseG1GC

```
-XX:MaxGCPauseMillis=200
-Xms512m
-Xmx1024m
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/tmp/heapdump.hprof
```

These help optimize memory and collect dumps automatically during failure.

---

#### ◆ Real-Time Production Practice

##### 1. Enable GC Logs

```
-Xlog:gc*,gc+heap=info:file=gc.log:time,uptime,level,tags
```

##### 2. Use Prometheus Alerts

Alert when heap usage > 85%

Alert when GC time > 10%

##### 3. Auto-Restart with Kubernetes Liveness Probe

Restart container if memory leak detected

Q3: Are singleton beans thread-safe? Not automatically — developer must ensure thread safety when using shared mutable state.

Q4: Can we make a prototype bean inside a singleton bean? Yes, but you must use `@Lookup` or `ObjectFactory` to get a new instance every time.

Q5: How to change the scope of a bean? Use `@Scope("prototype")`, `@Scope("request")`,

#### ◆ What is ELK Stack?

ELK stands for Elasticsearch, Logstash, and Kibana.

It is a powerful logging and monitoring solution widely used in Spring Boot microservices for analyzing logs in real time.

---

#### ⚙️ ELK Components Overview

| Component | Description | Spring Boot Integration |
|-----------|-------------|-------------------------|
|-----------|-------------|-------------------------|

|               |                                                                                                 |  |
|---------------|-------------------------------------------------------------------------------------------------|--|
| Elasticsearch | Stores and indexes logs for fast searching. Logs are pushed to Elasticsearch as JSON documents. |  |
|---------------|-------------------------------------------------------------------------------------------------|--|

Logstash Processes and transforms logs before storing them. Collects logs from files or directly via TCP/HTTP input.

Kibana UI for visualizing logs and metrics. Developers and DevOps can search, filter, and create dashboards for logs.

---

## 💡 Real-Time Usage in Spring Boot

### ♦ 1. Centralized Log Management

Instead of searching logs in each microservice, you can send all logs to Logstash → Elasticsearch → Kibana for centralized monitoring.

Example flow:

Spring Boot App → Logstash → Elasticsearch → Kibana

### ♦ 2. JSON Logging

Use Logback or Log4j2 to format logs in JSON, making them compatible with ELK.

Example: logback-spring.xml

```
<configuration>
  <appender name="JSON_FILE" class="ch.qos.logback.core.FileAppender">
    <file>logs/app.json</file>
    <encoder class="net.logstash.logback.encoder.LoggingEventCompositeJsonEncoder"/>
  </appender>

  <root level="INFO">
    <appender-ref ref="JSON_FILE"/>
  </root>
</configuration>
```

---

### ♦ 3. Sending Logs to Logstash

Configure Logstash TCP Appender in your Spring Boot app:

```
<appender name="LOGSTASH"
class="net.logstash.logback.appender.LogstashTcpSocketAppender">
  <destination>localhost:5000</destination>
  <encoder class="net.logstash.logback.encoder.LoggingEventEncoder"/>
</appender>

<root level="INFO">
```

```
<appender-ref ref="LOGSTASH"/>
</root>
```

Then, in Logstash configuration (logstash.conf):

```
input {
  tcp {
    port => 5000
    codec => json_lines
  }
}
output {
  elasticsearch {
    hosts => ["localhost:9200"]
    index => "springboot-logs"
  }
}
```

Now your Spring Boot logs appear in Kibana.

---

## ELK Stack Features in Real Time

### Feature Description

-  Centralized Logging Collect all microservice logs in one place.
-  Real-Time Search Elasticsearch lets you query logs instantly.
-  Visual Dashboards Kibana helps monitor traffic, errors, latency visually.
-  Error Pattern Detection Helps identify recurring issues or performance bottlenecks.
-  Alerting Can set up alerts for specific log patterns (e.g., "NullPointerException").
-  Audit & Security Logs Useful for compliance, tracking user/system activities.

---

### Real-Time Example Scenario

#### Use Case:

You have 10 microservices (Auth, User, Order, Payment, etc.).

Each service writes logs → Logstash → Elasticsearch → Kibana dashboard.

If Order Service fails, Kibana shows the stack trace.

You can filter logs by service name, level (ERROR/WARN), or traceId.

Combine with Spring Sleuth + Zipkin for distributed tracing.

---

## 💬 Interview Questions and Answers

① What is ELK Stack and why do we use it in Spring Boot?

Answer:

ELK (Elasticsearch, Logstash, Kibana) is used for centralized logging and monitoring in microservice environments. It helps developers easily search, analyze, and visualize logs from multiple Spring Boot applications in real time.

---

② How do you send logs from Spring Boot to ELK?

Answer:

By configuring Logback or Log4j2 with a Logstash appender that sends JSON logs over TCP/HTTP to the Logstash server, which then pushes them into Elasticsearch.

---

③ Why prefer JSON logs for ELK?

Answer:

JSON logs are structured and easily parsed by Logstash and Elasticsearch. They make it easier to filter and analyze logs using fields like timestamp, serviceName, level, message.

---

④ What's the difference between Logstash and Elasticsearch?

Answer:

Logstash: Collects, filters, and transforms logs.

Elasticsearch: Stores and indexes logs for fast searching and retrieval.

---

⑤ How do you visualize or monitor logs?

Answer:

Using Kibana, which provides dashboards, search queries, and filters to visualize logs and detect issues quickly.

---

⑥ How can you correlate logs between microservices?

Answer:

Use Spring Cloud Sleuth to add a traceId and spanId to each log entry. You can then search by traceId in Kibana to view the complete request flow across services.

---

#### 7 What are the challenges of using ELK?

Answer:

High storage cost for large log volumes

Logstash can become a bottleneck

Requires good indexing strategy

Needs maintenance for scaling Elasticsearch clusters

---

#### 8 How can you secure ELK Stack?

Answer:

Use HTTPS and Basic Auth on Kibana and Elasticsearch

Restrict access using role-based access control (RBAC)

Integrate with Keycloak or OAuth2 for authentication

How does Logstash handle unstructured logs? A: Using Grok filters to extract fields from plain text.

Q: How do you handle multiline logs (like Java stack traces)? A: Use the multiline codec in Logstash to group lines of the same exception.

---

#### ◆ What is Prometheus + Grafana?

Tool Purpose

Prometheus Time-series metrics collection and alerting system — it pulls data (metrics) from your Spring Boot app at regular intervals.

Grafana Visualization tool that creates dashboards and alerts using data from Prometheus (and other sources).

---

## ⚙️ How They Work Together

Flow:

Spring Boot App → Exposes metrics via /actuator/prometheus



Prometheus → Scrapes metrics periodically



Grafana → Reads Prometheus data → Shows dashboards

---

### ♦ 1. Spring Boot Integration (Real-Time Setup)

#### Step 1 Add Dependencies

In your pom.xml:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

---

#### Step 2 Enable Prometheus Endpoint

In application.yml:

```
management:
  endpoints:
    web:
      exposure:
        include: "prometheus,health,metrics"
  endpoint:
    prometheus:
      enabled: true
```

Now your app exposes metrics at:

<http://localhost:8080/actuator/prometheus>

This endpoint returns metrics like:

```
# HELP jvm_memory_used_bytes The amount of used memory
# TYPE jvm_memory_used_bytes gauge
jvm_memory_used_bytes{area="heap",id="PS Eden Space",} 3.445678E7
```

---

### Step 3 Configure Prometheus

In prometheus.yml:

```
global:
  scrape_interval: 5s # every 5 seconds

scrape_configs:
  - job_name: 'spring-boot-app'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['localhost:8080']
```

Run Prometheus (prometheus.exe or Docker), and it will scrape metrics automatically.

---

### Step 4 Connect Grafana

1. Open Grafana UI (default: <http://localhost:3000>)
2. Go to → Connections → Data Sources → Add Prometheus
3. Set URL: <http://localhost:9090>
4. Create a new dashboard with panels showing metrics like:

CPU usage

Memory

HTTP request latency

Error rate

---



## ◆ 2. Real-Time Features

### Feature Description

 Time-Series Metrics Prometheus stores metrics (like CPU, memory, API response time) over time.

 Alerting Prometheus Alertmanager can trigger alerts (email, Slack, etc.) when thresholds are crossed.

 Dynamic Dashboards Grafana visualizes Prometheus data in beautiful real-time graphs.

 Pull Model Prometheus pulls metrics from apps — easier to scale and secure.

 Micrometer Integration Spring Boot uses Micrometer to export metrics to Prometheus.

 Multi-Service Monitoring Each microservice can have its own metrics endpoint; Prometheus can scrape all.

---

### Real-Time Example Scenario

Use Case:

You have 5 microservices — Auth, User, Order, Payment, Notification.

Each one:

Exposes /actuator/prometheus

Prometheus scrapes them every 5s

Grafana dashboard shows:

API response time (http\_server\_requests\_seconds)

Error rate (http\_server\_requests\_seconds\_count{status="500"})

JVM heap usage

Active DB connections

When latency in Order Service spikes, Grafana's alert notifies the DevOps team.

---

### Common Metrics You Get from Spring Boot

#### Metric Name Description

http\_server\_requests\_seconds\_count Number of HTTP requests

http\_server\_requests\_seconds\_sum Total request duration

jvm\_memory\_used\_bytes Memory used by JVM

process\_cpu\_usage CPU usage of the app  
tomcat\_sessions\_active\_current Active user sessions  
datasource\_connections\_active Database connection count

---

### Alerting Example (Prometheus)

Create a rule in alert.rules:

```
groups:
- name: springboot-alerts
  rules:
  - alert: HighErrorRate
    expr: rate(http_server_requests_seconds_count{status="500"}[1m]) > 5
    for: 1m
    labels:
      severity: critical
    annotations:
      summary: "High error rate detected"
```

If more than 5 errors occur per minute, Prometheus will trigger an alert.

---

### Prometheus vs ELK (Comparison)

Feature Prometheus + Grafana ELK Stack

|                    |                       |                        |
|--------------------|-----------------------|------------------------|
| Type               | Metrics (Performance) | Logs (Debugging)       |
| Data               | Numeric time-series   | Text/JSON logs         |
| Best for           | Monitoring, alerting  | Troubleshooting, audit |
| Storage            | Time-series DB        | Elasticsearch index    |
| Visualization      | Grafana               | Kibana                 |
| Spring Integration | Micrometer            | Logback + Logstash     |

 In real time, companies use both together:

> Prometheus + Grafana for monitoring health & performance  
ELK for detailed log analysis

---

### Interview Questions and Answers

 1) What is Prometheus used for in Spring Boot?

Answer:

Prometheus is used to collect and store time-series metrics from Spring Boot applications — like CPU usage, response time, and memory — for monitoring and alerting.

---

## ② How does Prometheus collect metrics?

Answer:

Prometheus uses a pull model — it scrapes metrics from endpoints like `/actuator/prometheus` exposed by your app via Micrometer.

---

## ③ What is the role of Grafana?

Answer:

Grafana connects to Prometheus and visualizes metrics in real time with custom dashboards and alerts.

---

## ④ What is Micrometer in Spring Boot?

Answer:

Micrometer is the metrics facade in Spring Boot — it collects application metrics and sends them to monitoring systems like Prometheus, Datadog, or CloudWatch.

---

## ⑤ What's the difference between ELK and Prometheus-Grafana?

Answer:

ELK → log analysis and debugging

Prometheus-Grafana → performance monitoring and alerting

Both are often combined for complete observability.

---

## ⑥ How can you set alerts?

Answer:

Define alerting rules in Prometheus or use Grafana's built-in alert system to notify via email, Slack, or PagerDuty.

---

## 7 What is “scraping” in Prometheus?

Answer:

Scraping means periodically fetching metrics from defined targets (like /actuator/prometheus) and storing them in Prometheus.

What is EntityManager?

EntityManager is the main interface used in JPA to interact with the database.

It is responsible for managing the lifecycle of entities — like creating, reading, updating, and deleting (CRUD).

---

- ♦ Q1: What is the Saga Pattern in microservices or distributed systems?

Answer: The Saga Pattern is a design pattern used to maintain data consistency across multiple microservices in a distributed transaction — without using a traditional two-phase commit (2PC). It breaks a long transaction into a series of smaller local transactions, each handled by one service.

If one transaction fails, a series of compensating transactions are triggered to undo the previous successful operations.

---

- ♦ Q2: Why do we need the Saga Pattern?

Answer: In a distributed system, each service has its own database and transaction boundary. Traditional transactions (ACID) can't span across services because of scalability and coupling issues.

So, the Saga Pattern ensures eventual consistency between services by using asynchronous coordination and rollback logic.

---

- ♦ Q3: How does the Saga Pattern work in real time?

Answer: Imagine an e-commerce order service:

1. Order Service → Creates a new order (local transaction).
2. Payment Service → Deducts the payment.
3. Inventory Service → Reduces the stock.

4. Shipping Service → Prepares shipment.

✓ If all succeed → order completes.

✗ If Inventory fails → Saga triggers compensating actions:

Refund payment (Payment Service)

Cancel order (Order Service)

Thus, the system remains eventually consistent.

---

♦ Q4: What are the types of Saga Patterns?

Answer: There are two main types:

1. Choreography-based Saga:

Each service produces and listens to events.

No central coordinator.

Example:

Order Service emits OrderCreatedEvent.

Payment Service listens → processes payment → emits PaymentCompletedEvent.

Inventory listens → reduces stock.

If any service fails, it emits a compensating event like PaymentRefundEvent.

2. Orchestration-based Saga:

A central coordinator (Saga Orchestrator) controls the flow.

It sends commands to each service and decides next actions based on responses.

Example: The orchestrator calls Payment, Inventory, and Shipping sequentially.

---

♦ Q5: How is Saga implemented in real projects?

Answer: In real-world systems:

Choreography Saga → uses event brokers like Kafka, RabbitMQ, or AWS SNS/SQS.

Orchestration Saga → uses workflow engines like Camunda, Temporal, or Axon Framework.

Each local transaction is persisted using local DB commit.

Compensating transactions are published as rollback events.

---

♦ Q6: What are advantages of Saga Pattern?

- ✓ Maintains data consistency without distributed locks.
- ✓ Highly scalable — no blocking between services.
- ✓ Decoupled services — can evolve independently.

---

♦ Q7: What are challenges of Saga Pattern?

- ⚠ Complex to implement (especially compensation logic).
- ⚠ Requires idempotent operations to avoid duplicates.
- ⚠ Debugging is harder since it's asynchronous.
- ⚠ Event ordering and failure handling must be well managed.

---

♦ Q8: Real-Time Example in Banking

When transferring money between two accounts:

Debit from Account Service A

Credit to Account Service B

If credit fails, a compensating transaction will refund the amount to Service A.  
This ensures eventual consistency instead of atomic commit.

---

♦ Q9: Difference Between Saga and Two-Phase Commit (2PC)

Feature Saga Pattern Two-Phase Commit

|             |                           |                      |
|-------------|---------------------------|----------------------|
| Coupling    | Loosely coupled           | Tightly coupled      |
| Scalability | Highly scalable           | Low scalability      |
| Rollback    | Compensating transactions | Rollback at DB level |

Coordinator Optional (Orchestrator-based) Required  
Use Case Microservices Monolith / tightly coupled systems

---

- ◆ Q10: How to monitor or log Saga transactions?

In real-time, you can:

Use distributed tracing (e.g., Zipkin, Jaeger).

Use centralized logs (ELK Stack, Grafana Loki).

Maintain a Saga log table for transaction states (STARTED, COMPLETED, COMPENSATED).

---

### Real-Time Concept Summary

In a microservices architecture, the API Gateway acts as the single entry point for all client requests.

Integrating Spring Security in the API Gateway helps to:

Authenticate users (using JWT / OAuth2 / Keycloak / Okta)

Authorize access before routing the request to backend microservices

Offload security logic from all microservices to one centralized layer

---

### Real-Time Interview Questions & Answers

---

Q1. What is the purpose of integrating Spring Security with an API Gateway?

A:

To provide a centralized security layer that handles authentication and authorization for all incoming client requests before they reach the backend microservices.

This ensures every request is validated and unauthorized users are blocked early.

---

Q2. How does Spring Security work inside the API Gateway?

A:

When a request arrives:

1. Spring Security filters intercept it.
2. The JWT or OAuth2 token in the header is validated.
3. If valid → the user is authenticated and request is routed.
4. If invalid → the gateway returns 401 Unauthorized without calling backend services.

---

Q3. What are the steps to integrate Spring Security in Spring Cloud Gateway?

A:

1. Add dependencies:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

2. Configure JWT validation in application.yml:

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://auth-server.com
```

3. Create a SecurityConfig class:

```
@Configuration
public class SecurityConfig {
    @Bean
    SecurityWebFilterChain springSecurityFilterChain(ServerHttpSecurity http) {
        return http
            .csrf().disable()
            .authorizeExchange(exchanges -> exchanges
                .pathMatchers("/auth/**").permitAll())
```



```

        .anyExchange().authenticated()
    )
    .oauth2ResourceServer(ServerHttpSecurity.OAuth2ResourceServerSpec::jwt)
    .build();
}
}

```

4. Use the token to forward requests to backend services.

---

Q4. What type of authentication is commonly used with gateways?

A:

JWT (JSON Web Token) – stateless, lightweight, widely used.

OAuth2 / OpenID Connect – for enterprise-level SSO (Keycloak, Okta, Cognito).

---

Q5. How does the Gateway forward authentication info to microservices?

A:

After token validation, the Gateway:

Forwards the original JWT token to downstream microservices in the Authorization header.

Or, it can extract user details (like username/roles) and pass them as custom headers.  
The backend microservices can re-validate or trust the gateway depending on the design.

---

Q6. Why is it better to handle security at the gateway instead of in each microservice?

A: ☒ Centralized authentication logic  
☒ Less duplicate code  
☒ Easier to manage tokens and permissions  
☒ Consistent security policies across all microservices  
☒ Reduced attack surface (gateway acts as shield)

---

Q7. How do you handle role-based access (RBAC) in gateway?

A:

In Spring Security, configure role-based authorization using `authorizeExchange()`:

```
.authorizeExchange()  
  .pathMatchers("/admin/**").hasRole("ADMIN")  
  .pathMatchers("/user/**").hasAnyRole("USER","ADMIN")  
  .anyExchange().authenticated()
```

---

Q8. What happens if the token is expired or invalid?

A:

Spring Security automatically intercepts and returns:

401 (Unauthorized) → for invalid or expired tokens

403 (Forbidden) → for valid token but insufficient permissions

The request never reaches backend microservices.

---

Q9. How do you test the integration?

A:

1. Generate a valid JWT token from the auth server.

2. Call the gateway endpoint using Postman with header:

Authorization: Bearer <token>

3. Verify:

Valid token → request passes to microservice

Invalid/expired token → 401 returned immediately

---

Q10. What are common real-time tools used with this setup?

A:

Keycloak / Okta / Cognito – authentication provider

Spring Cloud Gateway – gateway layer

## Spring Security / OAuth2 Resource Server – validation and authorization

JWT – token mechanism

---

### Real-Time Flow Diagram

Client → API Gateway (Spring Security + JWT Validation)  
→ Authenticated → Microservice 1 / Microservice 2 / ...  
→ Unauthorized → 401 / 403 Response

---

### Key Takeaways

Gateway + Spring Security = centralized authentication/authorization

JWT or OAuth2 used for stateless token validation

Security logic removed from microservices → cleaner architecture

Gateway ensures only valid requests reach backend services

Keycloak is an open-source identity provider used with Spring Security to handle authentication and authorization.

It issues JWT or OAuth2 tokens, which Spring Security validates to secure APIs and manage user access without building custom login logic.

---

### What is Logging?

Logging is the process of recording application events, messages, and errors into files or monitoring systems for debugging, analysis, and audit.

---

### How Logging Works in Real-Time

1. Application Event Occurs:

Example — user logs in, API request is made, DB query fails, etc.

2. Logger Captures Message:

A logging framework (like SLF4J, Logback, Log4j2) records the event message with context

(timestamp, thread, class name, level).

### 3. Log Storage:

Logs are written to a file, console, database, or cloud monitoring service (e.g., ELK, Splunk, CloudWatch).

### 4. Monitoring and Alerting:

Logs are aggregated and visualized to detect patterns, performance issues, or failures.

---

## Logging Features in Real-Time Applications

### Feature Description Example

Log Levels Defines importance (TRACE, DEBUG, INFO, WARN, ERROR) `logger.error("DB connection failed");`

Contextual Logging (MDC) Adds user/session/request info automatically `userId=123, traceId=XYZ`

Log Rotation Automatically splits or archives large log files Keeps logs manageable

Centralized Logging Collect logs from multiple microservices into one place ELK, Graylog, Splunk

Structured Logging (JSON) Easier to parse for analysis Useful in microservices

Log Correlation / Trace ID Track a request across services `traceId=abc123`

Masking Sensitive Data Hide passwords, tokens, personal info Security compliance

Async Logging Improves performance by writing logs in background `Log4j2 async appender`

---

## Real-Time Logging Setup Example (Spring Boot)

### Step 1: Add SLF4J and Logback (default in Spring Boot)

You can use the logger like this:

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.*;
```

```
@RestController
```

```
public class OrderController {
```

```
    private static final Logger logger = LoggerFactory.getLogger(OrderController.class);
```

```
    @GetMapping("/order/{id}")
```

```
    public String getOrder(@PathVariable String id) {
        logger.info("Received request for order: {}", id);
    }
}
```

```

    try {
        // simulate logic
        int result = 10 / 0;
    } catch (Exception e) {
        logger.error("Error while fetching order {}: {}", id, e.getMessage());
    }
    return "Order ID: " + id;
}
}

```

---

### ✅ Step 2: Customize application.yml

```

logging:
  level:
    root: INFO
    com.example.order: DEBUG
  file:
    name: logs/order-service.log
  pattern:
    console: "%d{yyyy-MM-dd HH:mm:ss} %-5level [%thread] %logger{36} - %msg%n"

```

---

### ✅ Step 3: Use MDC for Trace Logging (Microservices)

```

import org.slf4j.MDC;

public void processRequest(String userId) {
    MDC.put("userId", userId);
    logger.info("Processing user request");
    MDC.clear();
}

```



Output:

```
2025-10-19 12:03:15 INFO [main] userId=123 - Processing user request
```

---

### 🧩 Real-Time Use Cases

#### Use Case Logging Example

API Request Lifecycle Log request start, end, and response time  
 Exception Handling Log stack trace and context  
 Security Auditing Log login attempts, access failures  
 Microservice Tracing Log traceId or correlationId

Performance Tuning Log slow queries or API latency  
Production Monitoring Send logs to ELK/Splunk dashboards

---

## 💬 Interview Questions & Answers

Q1. What is logging and why is it important?

A:

Logging records runtime information about the application's behavior.  
It helps in debugging, monitoring, auditing, and understanding production issues.

---

Q2. What are different log levels?

A:

Level Purpose

TRACE Detailed information for debugging

DEBUG Developer-level info

INFO Application progress / major events

WARN Potential problems

ERROR Failures or exceptions

---

Q3. What logging framework does Spring Boot use?

A:

Spring Boot uses SLF4J (Simple Logging Facade for Java) with Logback as the default implementation.

You can switch to Log4j2 if needed.

---

Q4. What is MDC in logging?

A:

MDC (Mapped Diagnostic Context) adds contextual data (like userId, requestId, sessionId) to logs automatically.

Useful for debugging multi-threaded or distributed systems.

---

Q5. How do you track a request across multiple microservices?

A:

Use a Trace ID / Correlation ID.

Add it in each log statement or HTTP header and propagate it between services (common with tools like Sleuth or Zipkin).

---

Q6. What is centralized logging and why is it needed?

A:

Centralized logging collects all microservice logs into one place for search, monitoring, and analysis (using ELK, Graylog, Splunk).

It simplifies production debugging and alerting.

---

Q7. How do you avoid logging sensitive data?

A:

Use masking filters or avoid logging confidential fields like passwords, tokens, or PII.

E.g., "password=\*\*\*\*".

---

Q8. What's the difference between synchronous and asynchronous logging?

A:

| Type         | Description                   | Use Case                       |
|--------------|-------------------------------|--------------------------------|
| Synchronous  | Writes logs directly          | Simple apps                    |
| Asynchronous | Uses queue/thread for logging | High-performance microservices |

---

Q9. How do you manage log file size?

A:

Use log rotation and rolling policies in Logback or Log4j2.

Example:

```
<rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
  <fileNamePattern>logs/app-%d{yyyy-MM-dd}.log</fileNamePattern>
  <maxHistory>30</maxHistory>
</rollingPolicy>
```

---

Q10. How do you monitor logs in real time?

A:

By streaming logs to:

ELK Stack (Elasticsearch, Logstash, Kibana)

Splunk

AWS CloudWatch / Azure Monitor / GCP Logging

---

Q11. What's the difference between logging and monitoring?

A:

Logging Monitoring

Records events Observes system health

Raw data (text) Metrics and alerts

Reactive Proactive

Example: "DB error" Example: "CPU usage > 90%"

---

### Best Practices Summary

1. Use proper log levels (avoid DEBUG in production).
2. Use JSON logs for microservices.
3. Include traceId and userId for correlation.
4. Mask sensitive data.
5. Store logs centrally (ELK/Splunk).
6. Rotate logs to manage size.
7. Use async logging for performance.

DEBUG level is used to record detailed information useful for developers while diagnosing or troubleshooting issues.

It shows internal application flow, variable values, and method calls — mainly used in development, not in production.



---

## ⚙️ Why Should We Avoid DEBUG Logs in Production

### ♦ 1. Performance Overhead

DEBUG level logs generate a large amount of detailed data (method calls, variable values, etc.).

Writing these logs (especially to files or remote servers) slows down application performance.

Logging itself is I/O intensive — even if the log isn't written, string concatenation and formatting consume CPU time.

#### 🧠 Example:

If a microservice processes 1000 requests per second and logs 20 DEBUG lines per request → millions of logs per minute.

---

### ♦ 2. Storage Problems

DEBUG logs fill up log files very quickly.

This can exhaust disk space or cause log rotation issues in production.

Once the disk is full, the application or server may crash.

#### 🧠 Example:

A service writing DEBUG logs might generate GBs of logs daily, making log analysis tools (like ELK or Splunk) costly and slow.

---

### ♦ 3. Security Risks

DEBUG logs may contain sensitive information (e.g., tokens, credentials, payload data).

In production, exposing internal details could lead to data leaks or security breaches.

#### 🧠 Example:

If you log entire request objects during debugging → personal user data could end up in log files.

---

### ♦ 4. Noise in Logs (Hard to Debug Real Issues)

DEBUG logs create a lot of noise.

Makes it hard to quickly spot important WARN or ERROR logs in production.

Log monitoring tools (like Kibana or CloudWatch) get flooded with unnecessary information.

---

#### ◆ 5. Increased Log Processing Cost

In distributed systems, logs are sent to centralized systems (like ELK, Splunk, CloudWatch).

More logs → higher bandwidth, storage, and license cost (many tools charge per GB).

---

#### ✓ When DEBUG Can Be Used

Environment Recommendation

Local / Dev ✓ Use DEBUG freely for detailed troubleshooting

QA / Staging ⚠ Enable temporarily to debug environment-specific issues

Production ✗ Avoid — use INFO, WARN, ERROR only

If needed, enable DEBUG temporarily for specific components using configuration.

#### ✖ Example (Spring Boot):

logging:

level:

com.example.order: DEBUG # Enable only for one package

---

#### 💬 Interview-Style Answer Example

> Q: Why should you avoid DEBUG logs in production?

A:

In production, DEBUG logs can severely impact performance, consume excessive disk space, and expose sensitive data. They also make it harder to spot critical issues due to log noise. We generally use INFO, WARN, and ERROR levels in production and enable DEBUG only temporarily for troubleshooting specific modules.

---

#### ⚙ What is a Retry Mechanism?

A Retry Mechanism automatically re-attempts a failed operation after a short delay — useful when temporary network glitches, timeouts, or service unavailability occur.

👉 Example:

If your service calls a payment API and it fails due to a temporary network issue, the retry mechanism tries again after a delay before reporting a failure.

---

### 🧠 How Retry Mechanism Works

#### 1. Detect Failure:

When an API call or operation fails (e.g., due to timeout, connection refused, etc.).

#### 2. Wait (Backoff):

Wait for a specified interval (fixed or exponential backoff).

#### 3. Retry:

Re-attempt the operation for a configured number of times.

#### 4. Give Up:

If all retries fail, it returns an error or triggers a fallback.

---

### 🔧 Retry Features in Real-Time

#### Feature Description Example

Retry Count Number of times to re-attempt Retry 3 times

Backoff Policy Delay strategy (fixed/exponential/randomized) 1s, 2s, 4s...

Circuit Breaker Integration Stops retrying when a service is known to be down Avoids overload

Idempotency Ensures retrying doesn't cause duplicate effects POST → make it idempotent using unique request ID

Retry on Specific Exceptions Retry only for recoverable exceptions TimeoutException, 5xx errors

Fallback Mechanism Default behavior when retries fail Return cached data or default response

---

### 🏗️ Real-Time Example (Spring Boot)

#### Using Spring Retry:

```
import org.springframework.retry.annotation.Retryable;
import org.springframework.retry.annotation.Recover;
```

```

import org.springframework.stereotype.Service;

@Service
public class PaymentService {

    @Retryable(
        value = {RuntimeException.class},
        maxAttempts = 3,
        backoff = @Backoff(delay = 2000, multiplier = 2)
    )
    public String makePayment() {
        System.out.println("Trying payment...");
        // Simulate network error
        throw new RuntimeException("Payment gateway not responding");
    }

    @Recover
    public String recover(RuntimeException e) {
        System.out.println("Fallback: " + e.getMessage());
        return "Payment failed, try again later.";
    }
}

```

 Explanation:

maxAttempts = 3 → Retry 3 times

@Backoff(delay = 2000, multiplier = 2) → Delay doubles each time

@Recover → Called when all retries fail

---

 Real-Time Use Cases

Scenario Where Retry Helps

API Gateway calling microservice Retry when microservice is temporarily unavailable

Payment gateway integration Retry on network timeouts

Message queue consumer Retry message processing if temporary DB failure

File uploads Retry on partial upload failure

External service integration Retry when 3rd-party API is slow or intermittent

---

 Interview Questions and Answers

Q1. What is a retry mechanism and why is it needed?

A: Retry mechanism re-attempts failed operations automatically. It helps handle transient failures such as network issues, timeouts, or temporary service unavailability, improving reliability and resilience.

---

Q2. What is the difference between retry and circuit breaker?

A:

Retry Circuit Breaker

Re-attempts the operation after failure  
Stops requests when service is known to be down  
Handles transient failures  
Prevents overload on failing services  
Works on per-call basis  
Works globally for service health

They are often used together — retry before tripping a circuit breaker.

---

Q3. What are common retry strategies?

A:

1. Fixed delay: Retry every x seconds.
2. Exponential backoff: Delay doubles after each retry.
3. Randomized backoff (Jitter): Adds randomness to prevent load spikes.

---

Q4. How do you avoid duplicate operations during retry?

A: By using idempotent APIs — each request has a unique identifier (like a transaction ID) so repeated retries don't create duplicates.

---

Q5. What are the drawbacks of using retries?

A:

Can overload downstream services

Increases latency

May cause duplicate requests if not idempotent

Should be combined with circuit breaker and backoff policies

---

Q6. How is retry implemented in Spring Boot?

A: Using:

spring-retry module with `@Retryable` and `@Recover`

Resilience4j Retry module integrated via annotations or configuration

---

Q7. How do you configure retry in Resilience4j?

Example:

resilience4j.retry:

instances:

paymentService:

max-attempts: 3

wait-duration: 2s

retry-exceptions:

- [java.io.IOException](#)

- `java.util.concurrent.TimeoutException`

---

Q8. How do you decide when to retry and when not to?

A:

Retry only for transient or recoverable errors (like timeouts, 5xx).

Don't retry for client-side or validation errors (like 400 Bad Request).

---

Q9. What happens if retry fails even after all attempts?

A:

You can implement a fallback — return cached data, enqueue for later processing, or send an alert.

---

Q10. How do you test a retry mechanism in real-time?

A:

Simulate network failure or throw exceptions

Check if retries occur as configured (logs, counters)

Verify exponential delay and fallback behavior

---

### ✓ Key Takeaways

Retry improves resilience, not reliability.

Use backoff and circuit breakers together.

Ensure operations are idempotent.

Configure retries only for transient failures.

## 9. Tools Commonly Used in Real Projects

Purpose Tool/Tech Used

Service Discovery Eureka / Consul  
Gateway Spring Cloud Gateway / Zuul  
Config Management Spring Cloud Config Server  
Fault Tolerance Resilience4j / Hystrix  
Asynchronous Messaging Kafka / RabbitMQ  
Logging ELK Stack / Sleuth + Zipkin  
Deployment Docker / Kubernetes  
CI/CD Jenkins / GitHub Actions

### ① If one service fails, how will your system behave?

We use Circuit Breaker (Resilience4j) to stop calls to the failed service and return a fallback response, preventing cascading failures.

---

### ② How did you handle circular dependency between services?

By refactoring service responsibilities, introducing an event-driven approach (Kafka) or a shared communication service to remove direct dependencies.

---

③ What will you do if your Config Server goes down?

Services use Spring Cloud Config's fail-fast + retry + local cache (bootstrap.yml) so they continue running with last known configuration until server recovers.

---

④ How do you ensure API backward compatibility?

Use versioning (v1, v2) in endpoints and ensure new fields are optional to not break existing clients.

---

⑤ How do you handle versioning in microservices?

Maintain separate endpoints like /api/v1/..., /api/v2/... or use header-based versioning.

Old versions are supported until clients migrate.

---

⑥ How do you ensure that a failed service doesn't impact others?

Implement Circuit Breaker, Retry, Timeout, and Bulkhead patterns.

Use asynchronous messaging (Kafka) where possible to decouple services.

---

⑦ How do you test microservices end-to-end?

Use Postman / REST Assured / Karate for API testing.

Run integration tests with Docker Compose to start all dependent services.

---

⑧ If your Kafka broker goes down, how will the system behave?

Messages are temporarily queued or retried.

Producers use retry & backoff, consumers resume once broker is available.

Critical services switch to fallback logic or local storage temporarily.



---

⑨ How do you roll back a failed deployment of one microservice?

Use Blue-Green or Canary deployment strategy.

If new version fails, switch traffic back to the stable version immediately.

---

⑩ How do you detect latency or bottleneck in your system?

Use distributed tracing tools like Zipkin, Sleuth, or Jaeger and monitoring dashboards with Prometheus + Grafana to track slow endpoints or dependencies.

---

① Why do we prefer microservices over monoliths in real-time systems?

Microservices allow independent development, deployment, and scaling of each module.

Fault isolation is easier — one service failure doesn't bring down the whole system.

Enables technology flexibility and faster releases.

---

② What happens if your API Gateway itself fails?

We deploy multiple API Gateway instances behind a Load Balancer (like Nginx / AWS ALB).

So if one fails, traffic is automatically routed to another instance → ensures high availability.

---

③ How do you handle inter-service authentication without exposing tokens?

Use JWT tokens or OAuth2 with internal service-to-service authentication.

Services verify tokens using a shared secret or public key (asymmetric encryption) via Auth Server — tokens are validated, not re-shared.

---

④ Can two microservices share one DB? Why or why not?

✗ No, each microservice should have its own database to maintain loose coupling and data

autonomy.

Shared DB leads to tight coupling, deployment conflicts, and data integrity risks.

Instead, use events (Kafka) or APIs for data sharing.

---

⑤ What patterns do you use to achieve resilience (Circuit Breaker, Retry, Bulkhead)?

Circuit Breaker → Stop calls to failing service.

Retry → Reattempt temporary failed calls with backoff.

Bulkhead → Isolate thread pools so one service failure doesn't consume all resources.

Together they ensure fault tolerance and stability under load.

---

⑥ How did you implement distributed logging across microservices?

Used Spring Cloud Sleuth to add trace ID and span ID in each log.

Integrated with Zipkin / ELK Stack (Elasticsearch, Logstash, Kibana) for centralized viewing.

This helps trace a single request flow across multiple services easily.

---

① Why did you choose API Gateway in your project?

API Gateway acts as a single entry point for all client requests.

It handles routing, load balancing, authentication, rate limiting, logging, and fallbacks centrally.

Reduces complexity in microservices and improves security and performance.

---

② How does routing work in Spring Cloud Gateway?

Routing is predicate-based — requests are matched using conditions like Path, Method, Header, Host, etc.

Once matched, Gateway forwards the request to the respective microservice using route definitions in application.yml or Java configuration.

Example:

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: http://localhost:8082/
          predicates:
            - Path=/orders/**
```

---

### ③ What are filters in API Gateway and what types exist?

Filters are used to modify requests or responses before or after routing.

Two types:

Pre Filters → run before request is routed (for auth, logging, headers).

Post Filters → run after response is received (for modification, response logging).

Example: Add headers, validate tokens, handle response codes, etc.

---

### ④ How did you implement Circuit Breaker and Rate Limiter in Gateway?

Used Resilience4j Circuit Breaker to handle downstream failures and Redis RateLimiter to limit requests per user/IP.

Example:

```
filters:
  - name: CircuitBreaker
    args:
      name: paymentCB
      fallbackUri: forward:/fallback/payment
  - name: RequestRateLimiter
    args:
      redis-rate-limiter.replenishRate: 10
      redis-rate-limiter.burstCapacity: 20
      key-resolver: "#{@userKeyResolver}"
```

- ✓ CircuitBreaker → stops calls to failing services
- ✓ RateLimiter → controls request rate per user or IP

---

⑤ How do you handle global exception or fallback in Gateway?

Define a `GlobalFilter` or `@ControllerAdvice` to catch exceptions.

Configure `fallbackUri` in `CircuitBreaker` to return a custom fallback response when a service is unavailable.

Example:

```
@GetMapping("/fallback/payment")
public Mono<String> paymentFallback() {
    return Mono.just("Payment service is temporarily unavailable. Please try again later.");
}
```

---

⑥ Can API Gateway handle authentication? How?

✓ Yes.

Use a pre-filter to validate JWT tokens before forwarding requests.

Tokens are verified using public keys from Auth service.

Valid requests are routed; invalid ones return 401 Unauthorized.

This ensures centralized authentication at the gateway layer.

---

⑥ Deployment, CI/CD & Monitoring

① How do you deploy microservices (Docker / Kubernetes)?

Package each service as a Docker image.

Deploy images to Kubernetes cluster using Deployments and Services.

Use ConfigMaps and Secrets for configuration and secure data.

Scale services independently with replicas.

---

② How do you handle environment-specific configurations?

Use Spring Profiles (application-dev.yml, application-prod.yml).

Store sensitive configs in Config Server or Secrets Manager.

Services pick environment configs at runtime via profile selection.

---

### ③ What tools do you use for monitoring microservices?

Prometheus → collects metrics from services.

Grafana → visualizes metrics and dashboards.

ELK (Elasticsearch, Logstash, Kibana) → centralized logging and analysis.

Spring Boot Actuator → exposes endpoints for health, metrics, and traces.

---

### ④ How do you handle service logs and tracing?

Use Spring Cloud Sleuth to add traceId and spanId to logs.

Collect logs in ELK Stack for centralized access.

Use Zipkin / Jaeger for distributed tracing across services.

---

### ⑤ What is health check endpoint and how do you use it?

Exposed via Spring Boot Actuator (/actuator/health).

Returns service status: UP/DOWN.

Used by Load Balancers, Kubernetes liveness & readiness probes to detect unhealthy services.

---

### ⑥ How do you manage CI/CD for multiple microservices?

Use Jenkins / GitHub Actions / GitLab CI pipelines.

Build Docker images, run tests, push to registry, deploy to Kubernetes automatically.

Use separate pipelines for each microservice for independent deployment.

---

## ⑦ Resilience and Fault Tolerance

### ① What is Circuit Breaker and how do you implement it in Spring Boot?

Prevents repeated calls to failing service.

Implement with Resilience4j: define circuit breaker, fallback URI, and thresholds.

Stops cascading failures and improves system stability.

---

### ② What is Retry mechanism and how is it implemented?

Automatically retries failed calls for transient errors.

Implement with Resilience4j Retry or `@Retryable` in Spring Boot.

Configurable max attempts, backoff interval.

---

### ③ What is Bulkhead pattern and why is it used?

Isolates resources (thread pools, connections) per service or call.

Prevents one slow service from exhausting all resources, protecting other parts of the system.

---

### ④ How do you handle fallback responses?

Define fallback methods or endpoints in Circuit Breaker.

Returns cached or default response instead of failing completely.

Ensures graceful degradation.

---

### ⑤ What is Rate Limiting and how do you implement it?

Controls number of requests per user, IP, or service per time unit.

Implement in API Gateway (Redis RateLimiter) or in-service using bucket4j / Resilience4j.

---

## ⑥ How do you handle slow or unresponsive services?

Set timeouts for external calls (RestTemplate/WebClient).

Use Circuit Breaker to open if service is too slow.

Use asynchronous calls or message queues (Kafka/RabbitMQ) to decouple processing.

---

---

## ③ Communication Between Services

### ① What is the difference between synchronous and asynchronous communication?

Synchronous: Caller waits for response (e.g., REST API via RestTemplate/WebClient).

Asynchronous: Caller doesn't wait; response handled later (e.g., Kafka / RabbitMQ).

Async improves resilience and decoupling.

---

### ② How did you use RestTemplate / WebClient for communication?

RestTemplate: Synchronous HTTP calls.

WebClient: Non-blocking, reactive HTTP calls.

Example: `WebClient.get().uri("/orders").retrieve().bodyToMono(Order.class)`

---

### ③ How did you implement asynchronous communication (Kafka / RabbitMQ)?

Producer sends message to topic/queue.

Consumer listens and processes asynchronously.

Helps decouple services and handle spikes without blocking.

---

### ④ What happens when one microservice is down — how do others handle it?

Use Circuit Breaker + Fallback to prevent cascading failures.

Optionally, use message queues to queue requests until service recovers.

---

#### ⑤ How do you secure REST communication between microservices?

Use JWT tokens or OAuth2 for internal authentication.

Services verify token validity before processing.

Optionally use HTTPS and mTLS for encrypted internal communication.

---

#### ④ Database and Data Management

##### ① Each microservice should have its own database — why?

Ensures loose coupling and autonomy.

Prevents schema conflicts and enables independent deployments.

---

##### ② How do you ensure data consistency across multiple databases?

Use event-driven architecture (Kafka/RabbitMQ) for async updates.

Apply saga pattern or distributed transactions where necessary.

---

##### ③ What is eventual consistency and how do you handle it?

Data may temporarily differ across services but converges eventually.

Handle using events, retries, and compensating transactions.

---

##### ④ How do you handle transactions that span multiple services?

Use Saga pattern:

Each service executes local transaction.



On failure, compensation actions are triggered.

---

## ⑤ What strategies do you use for database migrations? (Flyway / Liquibase)

Use Flyway / Liquibase scripts versioned per microservice.

Automatically applied at service startup to ensure schema consistency.

---

## ⑤ Security & Authentication

### ① How did you implement authentication and authorization in your microservices?

Central Auth Service issues JWT tokens.

Microservices validate token and roles before processing requests.

---

### ② How does JWT authentication work in microservices?

Auth service generates JWT with user info and roles.

Microservices validate signature and extract claims for authorization.

No session storage needed; fully stateless.

---

### ③ What is OAuth2 and how does it differ from JWT?

OAuth2 is an authorization framework that lets one application access another application's resources on behalf of a user — for example, allowing a mobile app to access your Google Drive data using your Google account.

JWT (JSON Web Token) is a token format — a compact, secure way to represent user identity and claims.

Simple difference:

OAuth2 = Process (how you get the token).

JWT = Token (what you get and use after authorization).

---

④ How do you secure communication between services internally?

JWT / OAuth2 for service-to-service authentication.

Use HTTPS / mTLS to encrypt traffic.

Optionally use API Gateway as central security layer.

---

⑤ What is API Gateway's role in security?

Centralized authentication, authorization, token validation.

Blocks unauthorized requests before reaching services.

Can handle rate limiting and IP filtering as extra security.

---

⑥ How do you store and validate tokens across services?

Tokens are stateless (JWT) → no storage needed.

Services validate signature and expiration using shared secret or public key.

Optionally, maintain revocation list for critical invalidation.

---

① Microservices Architecture – Real-Time Questions

1. What is microservice architecture, and how is it different from monolithic architecture?

Microservices split a system into independent, deployable services.

Monolith is a single deployable unit.

Microservices allow independent scaling, fault isolation, and technology flexibility.

---

2. How do services communicate with each other in your project (Sync vs Async)?

Sync: REST/HTTP (WebClient/RestTemplate), caller waits for response.

Async: Kafka/RabbitMQ, message queues, caller doesn't wait.

Async improves resilience and decoupling.

---

3. How did you register and discover your services? (Eureka, Consul, etc.)

Used Eureka Server for service registration.

Each microservice registers at startup.

Clients query Eureka or use LoadBalancer to find service instances dynamically.

---

4. How do you handle service versioning?

Use endpoint versioning (/api/v1/orders) or header-based versioning.

Old versions are supported until clients migrate.

---

5. How do you design inter-service communication for reliability?

Use Circuit Breaker, Retry, Timeout, Bulkhead patterns.

Prefer async messaging for non-critical calls.

Ensure fallbacks for critical synchronous calls.

---

6. How do you handle data consistency across multiple microservices?

Event-driven architecture (Kafka/RabbitMQ) for async updates.

Use saga pattern or compensating transactions for critical workflows.

---

7. How do you handle transactions across microservices (saga, event-driven)?

Saga: Each service executes local transaction; on failure, triggers compensation actions.

Event-driven: Publish events and update services asynchronously to maintain eventual

consistency.

---

8. How do you deploy and scale microservices independently?

Deploy each service as Docker container.

Use Kubernetes to manage scaling (replicas) and service discovery.

Each service can scale independently based on load.

---

9. What are the challenges you faced when splitting a monolith into microservices?

Data separation and consistency issues.

Handling inter-service communication.

Managing deployment pipelines and distributed logging.

Increased operational complexity.

---

10. How do you ensure security between microservices? (JWT, OAuth2, API Gateway)

Use JWT tokens for stateless auth.

OAuth2 for authorization scopes.

API Gateway validates tokens before routing requests.

Optional HTTPS / mTLS for encrypted communication.

---

## 2 Spring Boot + Spring Cloud – Real-Time Questions

1. What is Spring Cloud, and how does it help in microservices architecture?

Provides tools for distributed systems: Service Discovery, Config Management, Circuit Breaker, Load Balancing, API Gateway.

Simplifies common microservices patterns.

---

2. Explain how API Gateway works in your system.

Acts as single entry point for clients.

Handles routing, authentication, rate limiting, logging, fallback.

Reduces complexity in individual services.

---

3. How do you configure routing and filtering in Spring Cloud Gateway?

Define routes in application.yml or Java config.

Use predicates (Path, Method, Header) for routing.

Filters (pre/post) for logging, auth, or modifying requests/responses.

---

4. What is a Circuit Breaker and how did you implement it? (Hystrix / Resilience4j)

Prevents repeated calls to failing services.

Implemented Resilience4j with fallback method or URI.

Protects system from cascading failures.

---

5. What is a Rate Limiter, and why do you use it?

Controls requests per user/IP/service.

Implemented in API Gateway (Redis RateLimiter) to prevent overload and abuse.

---

6. What is Service Discovery and how does it work in Spring Cloud?

Services register with Eureka/Consul at startup.

Clients query discovery server or use LoadBalancer to get healthy instances dynamically.

---

7. How do you configure centralized configuration management? (Spring Cloud Config Server)

Store config in Git or Config Server.

Services fetch configs at startup based on profiles.

Supports dynamic refresh using @RefreshScope.

---

8. How do you handle distributed tracing and logging? (Zipkin / Sleuth / ELK)

Spring Cloud Sleuth adds traceId/spanId.

Zipkin or Jaeger tracks requests across services.

Logs collected in ELK Stack for centralized analysis.

---

9. What is Load Balancing and how does Ribbon/Spring Cloud LoadBalancer work?

Distributes requests across multiple service instances.

Ribbon uses client-side load balancing (round-robin, random).

Spring Cloud LoadBalancer is the new standard for Spring Boot 3.x+.

---

10. How do you handle retries and fallbacks between services?

Use Resilience4j Retry for transient failures.

Fallback method returns default response if service continues failing.

Ensures system stays responsive even when dependencies fail.

Excellent and very smart question 🙌 — this is a real interview-level doubt that shows deep understanding.

Let's clearly break down the difference between using Circuit Breaker, Load Balancer, and Rate Limiter in an API Gateway vs inside individual microservices.

---

## High-Level Concept

### Layer Purpose

API Gateway Acts as entry point for all external requests — applies global rules (security, routing, rate limiting, etc.)

Inside Microservice Handles internal resilience — ensures one service's failure doesn't break the rest

---

## Comparison: API Gateway vs Inside Application

### Feature In API Gateway In Microservice (Application) Key Difference

 Circuit Breaker Protects gateway from repeatedly calling a failing microservice. Protects a service from failing due to a slow/downstream dependency (e.g., DB, another microservice). Gateway-level breaker prevents external cascading; service-level breaker prevents internal cascading.

 Load Balancer Distributes traffic across multiple instances of the same microservice. (e.g., Round Robin, Ribbon, Eureka, etc.) Used internally if the service itself calls another replicated service or cluster (e.g., DB replicas, internal APIs). Gateway handles client-to-service load balancing; internal handles service-to-service balancing.

 Rate Limiter Controls how many requests clients can send to the system (e.g., per IP/user/token). Controls how many requests the service can handle internally or to its dependencies. Gateway limits incoming traffic; service-level limiter protects internal resources.

---

## Example for Each

### Circuit Breaker Example:

In Gateway:

If Payment Service is down → Gateway returns fallback → doesn't call it.

In Payment Service:

If Payment Service calls Bank API and Bank API is slow → Circuit breaker stops further calls to bank temporarily.

 Gateway CB → Protects front-end user experience

 Service CB → Protects internal system stability

---

### Load Balancer Example:

In Gateway:

Gateway → <http://payment-service> with 3 instances → routes requests evenly.  
(e.g., Spring Cloud LoadBalancer, Eureka discovery)

In Service:

Payment Service internally connects to 2 database replicas — load balanced.

✓ Gateway LB → Distributes client traffic

✓ Service LB → Distributes dependency traffic

---

🚦 Rate Limiter Example:

In Gateway:

Allow 100 requests/min per user or IP → protect system from overuse or DDoS.

In Service:

Limit how many external API calls or database writes it performs per second.

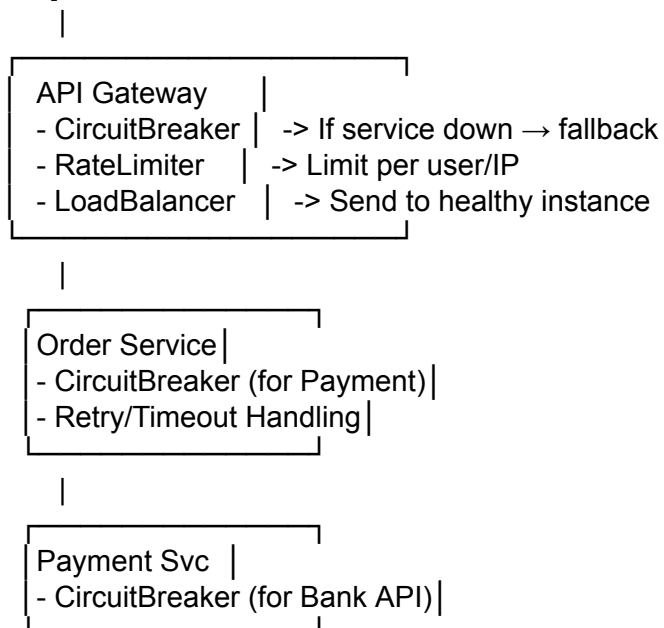
✓ Gateway RL → Protects entry point from abuse

✓ Service RL → Protects resource limits inside app

---

## 🧱 Real-Time Architecture View

[Clients]



---



## Why Use in Both Places?

Because failures can happen anywhere:

Gateway protects overall system stability and client-facing reliability.

Microservices protect their own dependencies and internal health.

So, both layers have their own responsibilities:

> Gateway → “Global Traffic Control”

Microservice → “Local Resilience”

---

## Summary Table

Concept   Gateway   Role   Microservice   Role

Circuit Breaker   Stop requests to failing services   Stop internal dependency calls

Load Balancer   Distribute client traffic   Distribute internal calls

Rate Limiter   Limit external users   Limit internal operations

---

Would you like me to give a diagram + example YAML + code setup showing how all three (CircuitBreaker + RateLimiter + LoadBalancer) work together in a Spring Cloud Gateway real project? That's often asked in senior-level interviews.

Software Engineer

Fresher

Hapag-Lloyd Software Engineer Interview Questions and Answers

Updated 6 Jun 2025

Overall Interview Experience Rating

4.3/5

based on 3 interview experiences

Difficulty level

Easy50%

Moderate50%

Duration

Less than 2 weeks100%

[View more](#)

5 Interview questions

A Software Engineer was asked

6mo ago

Q. What are the various mechanisms, apart from synchronized locks, to restrict multiple threads from making modifications in Java?

Ans. Java provides several mechanisms beyond synchronized locks to manage thread safety and restrict concurrent modifications.

1. ReentrantLock: A more flexible locking mechanism that allows for timed and interruptible locks. Example: `ReentrantLock lock = new ReentrantLock();`

2. ReadWriteLock: Allows multiple threads to read but only one to write, improving performance for read-heavy scenarios. Example: `ReadWriteLock rwLock = new ReentrantReadWriteLock();`

3. Atomic Variables: Classes like `AtomicInteger` and `AtomicReference` provide lock-free thread-safe operations. Example: `AtomicInteger atomicInt = new AtomicInteger(0);`

4. Concurrent Collections: Java provides thread-safe collections like `ConcurrentHashMap` and `CopyOnWriteArrayList`. Example: `ConcurrentHashMap<String, String> map = new ConcurrentHashMap`

About

Reviews

703

Salaries

2.9k

Interviews

29

Jobs

2

Benefits

72

Photos

-

Software Engineer

Fresher

Hapag-Lloyd Software Engineer Interview Questions and Answers

Updated 6 Jun 2025

Overall Interview Experience Rating

4.3/5

based on 3 interview experiences

Difficulty level

Easy50%

Moderate50%

Duration

Less than 2 weeks100%

View more

5 Interview questions

A Software Engineer was asked

6mo ago

Q. What are the various mechanisms, apart from synchronized locks, to restrict multiple threads from making modifications in Java?

Ans. Java provides several mechanisms beyond synchronized locks to manage thread safety and restrict concurrent modifications.

1. ReentrantLock: A more flexible locking mechanism that allows for timed and interruptible

locks. Example: `ReentrantLock lock = new ReentrantLock();`

2. `ReadWriteLock`: Allows multiple threads to read but only one to write, improving performance for read-heavy scenarios. Example: `ReadWriteLock rwLock = new ReentrantReadWriteLock();`

3. Atomic Variables: Classes like `AtomicInteger` and `AtomicReference` provide lock-free thread-safe operations. Example: `AtomicInteger atomicInt = new AtomicInteger(0);`

4. Concurrent Collections: Java provides thread-safe collections like `ConcurrentHashMap` and `CopyOnWriteArrayList`. Example: `ConcurrentHashMap<String, String> map = new ConcurrentHashMap<>();`

5. `ThreadLocal`: Provides thread-local variables, ensuring that each thread has its own instance. Example: `ThreadLocal<Integer> threadLocalValue = new ThreadLocal<>();`

Answered by AI

Add your answer

Share

A Software Engineer was asked

6mo ago

Q. What are the methods for implementing Interprocess Communication in a Microservice Architecture?

Ans. Interprocess Communication in microservices can be achieved through various methods like REST, messaging queues, and gRPC.

RESTful APIs: Services communicate over HTTP using JSON or XML. Example: A user service fetching data from an order service.

Message Queues: Asynchronous communication using tools like RabbitMQ or Kafka. Example: Order service sending messages to a payment service.

gRPC: High-performance RPC framework using Protocol Buffers. Example: A service calling another service's method directly with low latency.

Event Streaming: Services publish and subscribe to events. Example: A user service emitting an event when a user registers, which other services can listen to.

Software Engineer

Fresher

Hapag-Lloyd Software Engineer Interview Questions and Answers

Updated 6 Jun 2025

Overall Interview Experience Rating

4.3/5

based on 3 interview experiences

Difficulty level

Easy 50%

Moderate 50%

Duration

Less than 2 weeks 100%

View more

5 Interview questions

A Software Engineer was asked

6mo ago

Q. What are the various mechanisms, apart from synchronized locks, to restrict multiple threads from making modifications in Java?

Ans. Java provides several mechanisms beyond synchronized locks to manage thread safety and restrict concurrent modifications.

1. **ReentrantLock**: A more flexible locking mechanism that allows for timed and interruptible locks. Example: `ReentrantLock lock = new ReentrantLock();`
2. **ReadWriteLock**: Allows multiple threads to read but only one to write, improving performance for read-heavy scenarios. Example: `ReadWriteLock rwLock = new ReentrantReadWriteLock();`
3. **Atomic Variables**: Classes like `AtomicInteger` and `AtomicReference` provide lock-free thread-safe operations. Example: `AtomicInteger atomicInt = new AtomicInteger(0);`
4. **Concurrent Collections**: Java provides thread-safe collections like `ConcurrentHashMap` and `CopyOnWriteArrayList`. Example: `ConcurrentHashMap<String, String> map = new ConcurrentHashMap<>();`
5. **ThreadLocal**: Provides thread-local variables, ensuring that each thread has its own instance. Example: `ThreadLocal<Integer> threadLocalValue = new ThreadLocal<>();`

Answered by AI

Add your answer

Share

A Software Engineer was asked

6mo ago

Q. What are the methods for implementing Interprocess Communication in a Microservice Architecture?

Ans. Interprocess Communication in microservices can be achieved through various methods like REST, messaging queues, and gRPC.

**RESTful APIs**: Services communicate over HTTP using JSON or XML. Example: A user service fetching data from an order service.

**Message Queues**: Asynchronous communication using tools like RabbitMQ or Kafka. Example: Order service sending messages to a payment service.

**gRPC**: High-performance RPC framework using Protocol Buffers. Example: A service calling another service's method directly with low latency.

**Event Streaming**: Services publish and subscribe to events. Example: A user service emitting an event when a user registers, which other services can listen to.

**GraphQL**: Allows clients to request specific data from multiple services in a single query.

Example: A client fetching user and order data in one request.

Answered by AI

Add your answer

Share

Software Engineer Interview Questions Asked at Other Companies

asked in Capgemini

Q1. In a dark room, there is a box of 18 white and 5 black gloves. You are allo ... read more

View answers (530)

asked in AmbitionBox

Q2. What is the difference between a stack and a queue? Give an example where y ... read more

View answers (7)

asked in TCS

Q3. What is the reason that the Iterative Waterfall model was introduced?

View answers (9)

asked in TCS

Q4. Can you describe a challenging technical problem you faced and how you solv ... read more

View answers (20)

asked in Marvel Realtors

Q5. Given two queues, q1 and q2, where a background process copies elements from ... read more

[View answer \(1\)](#)

[View All](#)

A Software Engineer was asked

6mo ago

Q. How can multiple requests to modify specific rows be restricted using the DataJPA interface in Spring Data JPA?

Ans. Use optimistic locking and transaction management to restrict concurrent modifications in Spring Data JPA.

Implement optimistic locking using @Versi...read more

[Add your answer](#)

[Share](#)

A Software Engineer was asked

6mo ago

Q. What are the different types of JVM memory and their uses?

Ans. JVM memory is divided into several areas, each serving specific purposes for efficient memory management in Java applications.

Heap Memory: Used for...read more

[Add your answer](#)

[Share](#)

A Software Engineer was asked

6mo ago

Q. What is the purpose of the String Constant Pool in Java?

Ans. The String Constant Pool optimizes memory usage by storing unique string literals in Java.

Strings are immutable in Java, meaning once created, they...read more

[Add your answer](#)

[Share](#)

Hapag-Lloyd Software Engineer Interview Experiences

3 interviews found

Software Engineer Interview Questions & Answers

Anonymous

posted on 10 Apr 2025

Interview experience

4

Good

Difficulty level

Easy

Process Duration

Less than 2 weeks

Result

-

I appeared for an interview in Mar 2025, where I was asked the following questions.

Q1. What are the various mechanisms, apart from synchronized locks, to restrict multiple threads from making modificati...read more

Ans. Java provides several mechanisms beyond synchronized locks to manage thread safety and restrict concurrent modifications.

1. ReentrantLock: A more flexib...read more

Answered by AI

Add your answer

Q2. What are the methods for implementing Interprocess Communication in a Microservice Architecture?

Ans. Interprocess Communication in microservices can be achieved through various methods like REST, messaging queues, and gRPC.

RESTful APIs: Services communi...read more

Answered by AI

Add your answer

Q3. How can multiple requests to modify specific rows be restricted using the DataJPA interface in Spring Data JPA?

Ans. Use optimistic locking and transaction management to restrict concurrent modifications in Spring Data JPA.

Implement optimistic locking using @Version annotation in your entity class.

Use @Transactional annotation to manage transactions effectively.

Handle OptimisticLockingFailureException to manage conflicts gracefully.

Consider using pessimistic locking with EntityManager for critical sections.

Answered by AI

Add your answer

Q4. What are the different types of JVM memory and their uses?

Ans. JVM memory is divided into several areas, each serving specific purposes for efficient memory management in Java applications.

Heap Memory: Used for dynamic memory allocation for Java objects. Example: Objects created using 'new'.

Stack Memory: Stores method call frames, local variables, and references. Example: Primitive data types and object references.

Method Area: Contains class-level data such as class structures, method data, and constants. Example: Static variables.

Program Counter Register: Keeps track of the currently executing thread's instruction. Example: Points to the next instruction.

Native Method Stack: Used for native methods written in languages like C or C++. Example: JNI (Java Native Interface) calls.

Answered by AI

Add your answer

Q5. What is the purpose of the String Constant Pool in Java?

Ans. The String Constant Pool optimizes memory usage by storing unique string literals in Java. Strings are immutable in Java, meaning once created, they cannot be changed.

The String Constant Pool is a special memory area in the Java heap.

When a string literal is created, Java checks the pool first to see if it already exists.

If it exists, the reference to the existing string is returned; if not, a new string is created.

Example: String s1 = "Hello"; String s2 = "Hello"; s1 == s2 returns true because both reference the same object.

## Types of Memory in Java

### 1] Heap Memory

Purpose: Used to store objects and instance variables created using new.

Shared among all threads.

Managed by: Garbage Collector (GC)

Divided into:

Young Generation:

New objects are created here.

Includes Eden Space, Survivor 1 (S0), and Survivor 2 (S1).

Old Generation (Tenured Space):

Stores long-lived objects that survived multiple GC cycles.

Metaspace (from Java 8+):

Stores class metadata, method info, and static variables (replaced PermGen).

#### Example:

```
Employee e = new Employee(); // Object stored in Heap
```

---

### 2] Stack Memory

Purpose: Stores method calls, local variables, and references to objects in the heap.

Each thread has its own stack (not shared).

When a method is called, a stack frame is created; it's removed when the method ends.

Faster access compared to heap.

 Error: StackOverflowError when recursive calls exceed memory limit.

#### Example:

```
int x = 10; // stored in Stack
```

---

### ③ Metaspace (Method Area / Class Area)

Purpose: Stores class-level information, including:

Class metadata

Static variables

Method definitions

Constant pool

Shared among all threads

From Java 8, Metaspace replaced PermGen, and it grows dynamically using native memory.

---

### ④ PC (Program Counter) Register

Purpose: Keeps the address of the current instruction being executed by a thread.

Each thread has its own PC register.

---

### ⑤ Native Method Stack

Purpose: Used when Java interacts with native (non-Java) code, such as C/C++ through JNI (Java Native Interface).

Each thread has its own native stack.

---

## ■ Summary Table

Memory Type Stores Shared Between Threads Managed By

Heap Objects, instance variables  Yes Garbage Collector

Stack Method calls, local vars  No Each Thread

Metaspace Class info, static vars  Yes JVM

PC Register Current instruction  No Each Thread

Native Method Stack Native code info  No Each Thread



---

✅ In Short:

Heap → Objects

Stack → Local variables & method calls

Metaspace → Class data & static members

PC Register → Current instruction

Native Stack → Native code execution

Stack

Definition: Linear data structure using LIFO (Last In First Out).

Main operations: push(), pop(), peek().

Example: Undo feature, browser back button.

---

Queue

Definition: Linear data structure using FIFO (First In First Out).

Main operations: enqueue(), dequeue(), peek().

Example: Print queue, ticket booking line.

Common Reasons Why a Microservice Becomes Slow or Down

🌿 1. High Load or Traffic Spike

Too many concurrent requests → thread pool exhaustion.

CPU or memory usage spikes → service can't process new requests.

Common during peak usage hours or unpredicted traffic surges.

➡ Effect: High response time, timeouts, or “Service Unavailable” errors.

---

## 2. Downstream Service Slowness

A service depends on another microservice or database.

If that dependency is slow/unavailable → this service also becomes slow.

Example:

API Gateway → Order Service → Payment Service → Database

If Payment DB is slow, Order Service also slows down.

➡ Effect: Chain reaction → cascading delay or system crash.

---

## 3. Database Performance Issues

Heavy queries (no indexes, joins on large tables).

Connection pool exhaustion.

Deadlocks or long-running transactions.

➡ Effect: Slow responses or database connection timeouts.

---

## 4. Network Latency or Packet Loss

Unstable internal network between microservices.

Slow DNS resolution or routing problems in cloud environment.

➡ Effect: Increased API response time or request failures.

---

## 5. Memory Leaks or Garbage Collection (GC) Pauses

Poorly written code keeps unused objects in memory.

JVM pauses for long GC cycles.

➡ Effect: Gradual slowdown, then OutOfMemoryError → service crash.

---

## 6. Thread/Connection Pool Exhaustion

Service handles too many simultaneous requests without releasing threads properly.

HTTP clients or DB connections not closed.

➡ Effect: New requests wait indefinitely → timeout.

---

## 7. Third-Party API or External Dependency Failure

If service depends on external APIs (e.g., payment gateways, SMS services), their downtime affects your service.

➡ Effect: Service keeps retrying → increased latency.

---

## 8. Misconfiguration or Deployment Issues

Wrong environment variables, timeouts, or memory limits.

CI/CD deployment errors or incompatible versions.

➡ Effect: Service may start but crash under real load.

---

## 9. Hardware or Cloud Infrastructure Issues

Server node failure in cloud.

Network partition in Kubernetes cluster.

Disk I/O problems.

➡ Effect: Pods restart, requests drop, or nodes unreachable.

---

## 10. Infinite Loops or Code Bugs

Developer mistake: recursive call, infinite loop, or blocking I/O in async code.

➡ Effect: Service CPU 100% → becomes unresponsive.

---

## Summary Table

Cause Type Result

High load Performance Slow response / timeout

Database slowness Dependency Delay or error 500

Network latency Infrastructure Request drops

GC pause / memory leak Runtime Freeze or crash  
Third-party API down External Chain failure  
Config or code bug Logic Crash / retry storm

---

🔒 Why Circuit Breaker Helps Here:

When any of these happen 📌

- ➡ The Circuit Breaker detects repeated failures
- ➡ Opens the circuit and stops sending more requests to the broken service
- ➡ Returns a fallback response
- ➡ Prevents system-wide slowdown or crash

---

Would you like me to show a real-time flow diagram showing how circuit breaker isolates a slow or down service in microservice architecture? (useful for interviews or presentations)

What is a Circuit Breaker in API Gateway?

A Circuit Breaker is a fault-tolerance pattern that prevents your system from repeatedly calling a failing service.  
It acts like an electrical circuit breaker — if too many failures happen, it “opens” the circuit and stops sending requests to that service temporarily.

---

💡 Real-Time Working (Step-by-Step)

◆ Scenario:

You have:

API Gateway → routes requests to microservices.

One of the services (say, payment-service) is slow or down.

---

◆ Step-by-Step Working:

1. Normal (Closed State)

Gateway routes requests to payment-service.

All works fine.

## 2. Failure Detection (Half-Open → Open)

Gateway monitors responses (via Resilience4j/Hystrix).

If a threshold of failures (e.g., 50% of last 10 requests) is reached, it opens the circuit — meaning it will stop calling the failing service.

## 3. Open State (Protection Mode)

Requests are not sent to the failing service.

Instead, Gateway immediately returns a fallback response (custom message, cached data, etc.).

This prevents cascading failures and saves system resources.

## 4. Half-Open State (Recovery Check)

After a “cool-down” time (say, 10 seconds), Gateway sends a few trial requests to see if the service is healthy again.

## 5. Closed State (Recovered)

If trial requests succeed → circuit closes → normal routing resumes.

If fails again → remains open.

---

## ☀ Key Features of Circuit Breaker in API Gateway

### Feature Description

- 🧠 Failure Detection Monitors requests for exceptions, timeouts, or HTTP errors.
- ⚡ Automatic Fallback Returns a fallback response when service fails.
- 🔄 State Management Manages transitions: Closed → Open → Half-Open → Closed.
- ⌚ Timeout Handling Rejects slow requests to prevent thread blocking.
- 🔄 Retry/Recovery Optionally retries requests when circuit is half-open.
- 📊 Metrics & Monitoring Provides metrics like failure rate, call count, response time.
- 🔒 System Protection Prevents one service's failure from affecting the entire system.

---

## 🌱 Real-Time Example (Spring Cloud Gateway + Resilience4j)

```

spring:
  cloud:
    gateway:
      routes:
        - id: payment-service
          uri: http://localhost:8082/
          predicates:
            - Path=/payment/**
          filters:
            - name: CircuitBreaker
              args:
                name: paymentCB
                fallbackUri: forward:/fallback/payment

```

Controller (Fallback endpoint):

```

@RestController
public class FallbackController {
    @GetMapping("/fallback/payment")
    public Mono<String> paymentFallback() {
        return Mono.just("Payment service is currently unavailable. Please try again later.");
    }
}

```

Explanation:

If the payment-service is down or too slow, the Circuit Breaker will open.

API Gateway immediately returns fallback message instead of waiting.

---

### Real-Time Benefits

- ☒ Prevents cascading failures across microservices
- ☒ Improves response time for users during service downtime
- ☒ Provides graceful degradation instead of total failure
- ☒ Helps auto-heal after temporary issues

---

### In Short:

State Meaning Action

Closed Normal Allow all requests

Open Service unhealthy Block requests & return fallback

Half-Open Testing recovery Send limited requests

---

Would you like me to show a complete Spring Boot API Gateway + Circuit Breaker (Resilience4j) example project structure with dependencies and code flow? That will help you answer real-time interview questions too.

What is Rate Limiting?

Rate limiting means controlling how many requests a client (user, IP, or API key) can make within a specific time window — for example:

> “Allow only 5 requests per second per user.”

It protects your APIs from abuse, DoS attacks, and helps manage fair usage.

---

#### Types of Gateway Filter Factories

Here are some important built-in Gateway Filter Factories in Spring Cloud Gateway:

##### Filter Factory Purpose

AddRequestHeader Adds a header to the request.

AddResponseHeader Adds a header to the response.

AddRequestParameter Adds a request parameter.

RewritePath Rewrites part of the URL path.

RemoveRequestHeader Removes specific headers from the request.

RemoveResponseHeader Removes specific headers from the response.

RequestRateLimiter Limits the rate of requests (RateLimiter).

Retry Retries failed requests based on conditions.

CircuitBreaker Breaks the circuit if too many failures occur.

RedirectTo Redirects to another URI or HTTPS.

SetPath Changes the path of the request.

SetStatus Sets the HTTP status code of the response.

SetResponseHeader Replaces or sets a response header.

RewriteResponseHeader Modifies response headers with regex.

PreserveHostHeader Keeps the original Host header.

RequestHeaderToRequestUri Sets a header value as a new URI.

StripPrefix Removes parts of the request path.

RequestSize Limits the maximum request size.

ModifyResponseBody Modifies response body content.

ModifyRequestBody Modifies request body content.

---

## ⚙️ In Spring Boot (Spring Cloud Gateway)

Spring Cloud Gateway has a built-in RateLimiter filter, usually implemented using the RedisRateLimiter class.

### 🔗 Components involved:

1. GatewayFilter – applies rate limit to certain routes.
2. RateLimiter – defines how rate limiting works (e.g., RedisRateLimiter).
3. Redis – stores request counts and timestamps.

### 🔗 Resilience4j RateLimiter – How counts and timestamps are stored

Resilience4j RateLimiter internally maintains a sliding window of permits using in-memory counters and timestamps (no external DB).

It's handled by an internal class called RateLimiter.State, which keeps track of:

---

## 🏗️ How It Works (Step-by-Step in Real Time)

### 1. Incoming Request

Each request passes through the API Gateway route.

Example:

GET /api/users → hits API Gateway

### 2. Gateway Identifies the Requester

The Gateway identifies the key (e.g., IP address or user ID).  
This key is used to track request count in Redis.

### 3. RateLimiter Checks Redis

The RedisRateLimiter uses Redis to store:

The number of tokens left for that key.

The last timestamp when tokens were refilled.

It checks if this user still has tokens available.

### 4. Decision:



✅ Allowed: If tokens are available → the request is forwarded to the backend microservice.

❌ Rejected: If no tokens left → Gateway returns HTTP 429 (Too Many Requests).

## 5. Token Refill:

Tokens are automatically replenished over time (e.g., every second).

---

### ⚡ Real-Time Flow Example

Let's say:

```
redis-rate-limiter.replenishRate = 5 # 5 requests per second
redis-rate-limiter.burstCapacity = 10 # allows small bursts
```

In one second:

A client can send up to 10 requests instantly (burst).

After that, 5 tokens per second are refilled.

If a client exceeds that rate → next requests get HTTP 429.

---

### 🧠 YAML Configuration Example

```
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: http://localhost:8081
          predicates:
            - Path=/api/users/**
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 5
                redis-rate-limiter.burstCapacity: 10
                key-resolver: "#{@ipKeyResolver}"
```

Add the KeyResolver Bean:

@Bean

```
KeyResolver ipKeyResolver() {  
    return exchange -> Mono.just(  
        exchange.getRequest().getRemoteAddress().getAddress().getHostAddress()  
    );  
}
```

This means rate limit is applied per IP address.

---

## Redis Setup

Spring Cloud Gateway uses Redis to store request counts because it's:

Fast (in-memory)

Distributed (shared between multiple gateway instances)

Atomic (safe for concurrent updates)

So when multiple Gateway instances are running in real time (like in a cluster), all share the same rate limit data via Redis.

---

## In Real-Time Production Example

Imagine:

A mobile app sends API calls through your gateway.

Each user can call /api/orders max 5 times per second.

Flow:

1. User sends 10 requests in 1 second.
2. RedisRateLimiter sees:  
10 requests, only 5 tokens allowed.
3. Gateway processes first 5 → forwards them to microservice.
4. Next 5 requests → HTTP 429 "Too Many Requests".
5. After 1 second, Redis adds 5 more tokens.

---

### Response Example (When Limit Exceeds)

```
{
  "timestamp": "2025-10-18T10:00:00Z",
  "status": 429,
  "error": "Too Many Requests",
  "message": "Rate limit exceeded. Try again later."
}
```

---

### Advanced Customization

You can also:

Use user ID or API key instead of IP.

Create custom RateLimiter implementations (not only Redis).

Use Spring Boot Actuator metrics to monitor rate limit hits.

Integrate with Prometheus / Grafana for live tracking.

---

### Summary

#### Step Description

- 1 Request enters API Gateway
- 2 KeyResolver determines who sent it
- 3 RedisRateLimiter checks Redis for tokens
- 4 If tokens available → forward
- 5 If no tokens → 429 Too Many Requests
- 6 Tokens refill periodical

Basic to Advanced: Real-Time Interview Q&A

---

#### 1. What is rate limiting and why do we use it in API Gateway?

Answer: Rate limiting controls how many requests a client can make to your API in a specific time window.

It prevents abuse (e.g., DDoS attacks), ensures fair usage among users, and protects downstream microservices from overload.

Real-time example:

> In our e-commerce system, we allowed only 10 API requests per second per user to prevent cart service from crashing during flash sales.

---

2. How is rate limiting implemented in Spring Cloud Gateway?

Answer: Spring Cloud Gateway provides a `RequestRateLimiter` filter, which can be configured using a `RateLimiter` implementation.

The default is `RedisRateLimiter`, which uses Redis to store request counters and timing.

Core components:

`RequestRateLimiter` filter

`RedisRateLimiter` class

`KeyResolver` (to identify users or IPs)

---

3. What is the role of `KeyResolver` in rate limiting?

Answer: `KeyResolver` determines how the system identifies a client.

It returns a key (like IP address, user ID, or API key) for which rate limiting should apply.

Example:

```
@Bean
KeyResolver ipKeyResolver() {
    return exchange -> Mono.just(
        exchange.getRequest().getRemoteAddress().getAddress().getHostAddress()
    );
}
```

So, each IP gets its own rate limit counter in Redis.

---

4. What are `replenishRate` and `burstCapacity` in `RedisRateLimiter`?

Answer:

Property Description

replenishRate How many tokens are added per second (e.g., 5 requests/sec).  
burstCapacity Maximum tokens that can be accumulated — allows short bursts above the normal rate.

Example:

replenishRate: 5  
burstCapacity: 10

→ 10 requests can be sent instantly, after that only 5 tokens refill every second.

---

5. What happens when the rate limit is exceeded?

Answer: Spring Cloud Gateway returns an HTTP 429 (Too Many Requests) response automatically.

You can customize this response body using global filters or error handling logic.

Example response:

```
{  
  "status": 429,  
  "message": "Too many requests, please try again later."  
}
```

---

6. Why is Redis used for rate limiting?

Answer: Redis is fast, distributed, and atomic, making it ideal for managing counters in real time across multiple gateway instances.

Real-time benefit:

In a clustered environment (multiple gateway nodes), Redis ensures rate limits are shared across all instances.

---

7. Can we apply rate limiting per API instead of per user?

Answer: Yes 

You can configure RequestRateLimiter for specific routes in your application.yml or Java configuration.

Example:

routes:

- id: product-service
- uri: <http://localhost:8081>
- predicates:
  - Path=/api/products/\*\*
- filters:
  - name: RequestRateLimiter
  - args:
    - redis-rate-limiter.replenishRate: 3
    - redis-rate-limiter.burstCapacity: 5
    - key-resolver: "#{@userKeyResolver}"

Each route can have different rate limits based on business logic.

---

8. What happens if Redis goes down?

Answer: If Redis is unavailable, the rate limiter may:

Fail open (allow all requests) — default behavior.

Or fail closed (block all requests) — configurable.

You can customize this by catching Redis connection exceptions and defining fallback logic.

---

9. How can you test rate limiting locally?

Answer: You can use Postman or JMeter to send multiple requests quickly to your API Gateway route.

Example test:

Send 20 requests in 2 seconds.

Expected: first 10 pass, next 10 get HTTP 429.

---

10. What is the difference between rate limiting and throttling?

Feature Rate Limiting Throttling

Purpose Restrict number of requests in time Control speed or delay between requests

Response Rejects (HTTP 429) Queues or delays requests

Example 10 requests per second 1 request every 500ms

---

11. Can we create a custom rate limiter in Spring Cloud Gateway?

Answer: Yes, by implementing the RateLimiter interface and overriding:

isAllowed(String routeId, String id)

getConfigClass()

newConfig()

You can use databases, in-memory caches, or external APIs to define limits.

Example use case:

> A SaaS product where each customer tier (Free, Pro, Enterprise) has different API limits.

---

12. What metrics can we track for rate limiting?

Answer: You can monitor:

Number of requests allowed vs. rejected

Rate limit exceeded count

Redis key usage and expiry rate

Use Spring Boot Actuator, Prometheus, or Grafana dashboards to visualize this.

---

13. How do you apply different rate limits for different users?

Answer: Implement a custom KeyResolver that checks user roles or tokens.

Example:

```
@Bean
KeyResolver userPlanResolver() {
    return exchange -> {
        String userPlan = exchange.getRequest()
            .getHeaders().getFirst("X-Plan");
        return Mono.just(userPlan != null ? userPlan : "FREE");
    };
}
```

```
};  
}
```

Then assign different rate limits in YAML for each plan.

---

14. How does token bucket algorithm work behind RedisRateLimiter?

Answer: RedisRateLimiter uses the Token Bucket algorithm:

1. Each user has a bucket with limited tokens.
2. Every request consumes one token.
3. Tokens refill at a fixed rate over time.
4. If bucket empty → request is rejected (HTTP 429).

This ensures smooth traffic handling with burst tolerance.

---

15. What are some best practices for using rate limiting in production?

Answer: ☒ Use Redis cluster for high availability  
☒ Apply per-user or per-API key limits  
☒ Provide custom Retry-After headers in 429 responses  
☒ Monitor rate limit stats via metrics  
☒ Implement graceful degradation on limit breach

---

💡 Bonus Tip (Real Project Example)

In a banking API gateway, we applied:

5 requests/second for regular users

50 requests/second for admin users

Used JWT KeyResolver to identify roles from tokens

Logged all “429 Too Many Requests” events to ELK for fraud detection

Excellent and very practical question 🙌 —

Yes ☒, in Spring Cloud Gateway, you can absolutely use API path, user ID, API key, JWT token, or any custom logic inside the KeyResolver, not just the IP address.



Let's break it down clearly 📌

---

## 💡 1. Default Example — IP Based

You already know this:

```
@Bean
KeyResolver ipKeyResolver() {
    return exchange -> Mono.just(
        exchange.getRequest().getRemoteAddress().getAddress().getHostAddress()
    );
}
```

👉 Limits based on client IP — useful when users are not authenticated.

---

## ⚙️ 2. User ID Based KeyResolver

If your request contains a user ID in headers, query param, or JWT token, you can extract and use it as the key.

Example 1 — From HTTP Header

```
@Bean
KeyResolver userIdKeyResolver() {
    return exchange -> {
        String userId = exchange.getRequest()
            .getHeaders()
            .getFirst("X-User-Id");
        return Mono.just(userId != null ? userId : "anonymous");
    };
}
```

🧩 This will rate limit per user ID (not per IP).

---

Example 2 — From Query Parameter

```
@Bean
KeyResolver queryParamsResolver() {
    return exchange -> Mono.just(
        exchange.getRequest().getQueryParams().getFirst("userId")
    );
}
```

```
}
```

✓ Useful for testing simple APIs.

---

### Example 3 — From JWT Token (Most Common in Real Systems)

If your API uses JWT authentication, you can decode the token and extract user ID or role.

```
@Bean
KeyResolver jwtKeyResolver() {
    return exchange -> {
        String authHeader = exchange.getRequest().getHeaders().getFirst("Authorization");
        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            String token = authHeader.substring(7);
            String userId = JwtUtil.extractUserId(token); // custom util to decode JWT
            return Mono.just(userId);
        }
        return Mono.just("anonymous");
    };
}
```

✓ Real-time use:

Each logged-in user has their own limit, regardless of IP.

---

### ⚙️ 3. API Path Based KeyResolver

You can also apply limits per API endpoint (route-based rate limiting).

```
@Bean
KeyResolver apiPathKeyResolver() {
    return exchange -> Mono.just(
        exchange.getRequest().getPath().value()
    );
}
```

✓ Example:

/api/products → limited separately from /api/orders

---

### ⚙️ 4. Combine User + API Path

You can combine both for more fine-grained control.

```

@Bean
KeyResolver userAndApiKeyResolver() {
    return exchange -> {
        String userId = exchange.getRequest().getHeaders().getFirst("X-User-Id");
        String path = exchange.getRequest().getPath().value();
        return Mono.just(userId + ":" + path);
    };
}

```

✅ Example:

User 101 hitting /api/orders → separate from same user hitting /api/products.

---

## 🧠 5. Configure in application.yml

You can reference any of these resolvers like this:

```

spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: http://localhost:8081
          predicates:
            - Path=/api/users/**
          filters:
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 5
                redis-rate-limiter.burstCapacity: 10
                key-resolver: "#{@userIdKeyResolver}"

```

---

## 🚀 6. Real-Time Example Scenario

E-commerce API Gateway

User Type Limit

Guest (no token) 3 req/sec

Logged-in User 10 req/sec

Admin 50 req/sec

You can extract role from JWT, and apply limits dynamically by route or user type.

---

## ✓ Summary

Use Case Example KeyResolver Possible

IP-based ipKeyResolver() ✓ Default  
User ID-based userIdKeyResolver() ✓  
JWT Token-based jwtKeyResolver() ✓  
API Path-based apiPathKeyResolver() ✓  
Combined (User + Path) userAndApiKeyResolver() ✓

So you are not limited to IP address — you can fully customize KeyResolver for any key you want.

---

Would you like me to show a real working example (JWT-based KeyResolver + RedisRateLimiter + sample route config + sample test request)?  
It's the most common pattern used in real-time projects.

Let's keep it short and clear 🙌

---

## ⚙️ Resilience4j Bulkhead Module (Real-Time Working)

### 💡 Concept:

Like compartments in a ship — it isolates failures by limiting the number of concurrent calls to a service or component.

### 🔧 Real-Time Working:

You define a limit (e.g., max 10 concurrent calls to Payment Service).

Extra requests are queued or rejected immediately.

Prevents one slow or failing service from blocking the entire system.

### 🧠 Example (Spring Boot)

```
@Bulkhead(name = "paymentService", type = Bulkhead.Type.THREADPOOL)
```

```
public String callPayment() {
    return restTemplate.getForObject("http://payment/api/pay", String.class);
}
```

#### ✓ Features:

Isolates thread pools per service.

Prevents resource exhaustion.

Improves overall system stability.

---

#### 🕒 Resilience4j Timeout Module (Real-Time Working)

##### 💡 Concept:

Defines a maximum wait time for a remote call — if it exceeds the limit, it automatically times out.

##### 🔧 Real-Time Working:

Example: Order Service calls Payment Service.

If Payment Service doesn't respond in 2 seconds, the call is cancelled and fallback logic executes.

#### 🧠 Example (Spring Boot)

```
@TimeLimiter(name = "paymentService")
@Bulkhead(name = "paymentService")
public CompletableFuture<String> callPaymentAsync() {
    return CompletableFuture.supplyAsync(() ->
        restTemplate.getForObject("http://payment/api/pay", String.class)
    );
}
```

#### ✓ Features:

Prevents long waits on slow services.

Improves system responsiveness.

Can combine with Retry or CircuitBreaker for better resilience.

---

In short:

Bulkhead: Limits concurrent calls → isolates failures.

Timeout: Cancels slow calls → improves responsiveness.

Both modules work together in Resilience4j to make microservices fault-tolerant in real time.

Java junit microservices, jpa

Hi,

I'm currently seeking new opportunities as a Java Developer. I have over 4.1 years of experience in Java, Spring Boot, and microservices, working on scalable backend solutions and API integrations. I'm passionate about clean code, problem-solving, and learning new technologies. Looking forward to contributing my skills in a dynamic team.